



Classification IV: Alternative Methods and Summary

Lecture 8

肖升生 博士

Dr. Shengsheng Xiao

Email: Xiao.shengsheng@shufe.edu.cn

Classification

categorical
categorical
continuous
class

Tid	Refund	Marital Status	Taxable Income	Cheat
1	Yes	Single	125K	No
2	No	Married	100K	No
3	No	Single	70K	No
4	Yes	Married	120K	No
5	No	Divorced	95K	Yes
6	No	Married	60K	No
7	Yes	Divorced	220K	No
8	No	Single	85K	Yes
9	No	Married	75K	No
10	No	Single	90K	Yes

Training set



Learn classifier



Model



Predicted classes

Different Classifiers



Refund	Marital Status	Taxable Income	Cheat
No	Single	75K	?
Yes	Married	50K	?
No	Married	150K	?
Yes	Divorced	90K	?
No	Single	40K	?
No	Married	80K	?

Test set



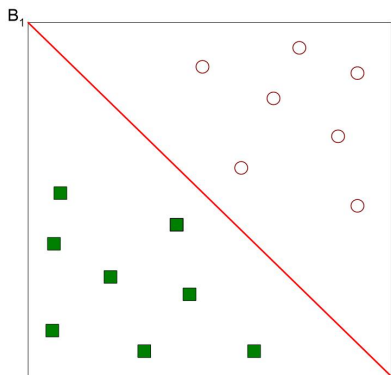


Outline

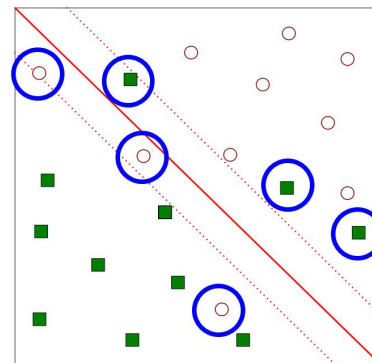
- Classification: Alternative Methods
 - SVM
 - ANN/Deep Learning
- Classification: Summarization
 - Loss Function
 - Underfitting and Overfitting
 - Model Evaluation

- Classification: Alternative Methods
 - SVM
 - ANN/Deep Learning
- Classification: Summarization
 - Loss Function
 - Underfitting and Overfitting
 - Model Evaluation

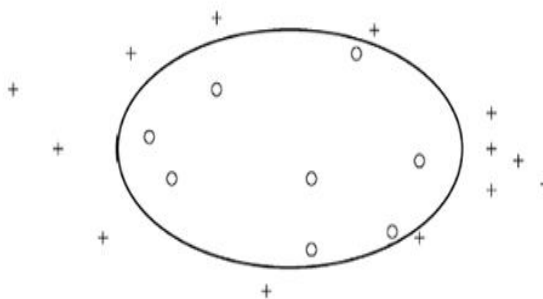
Support vector machines



linearly separable classes



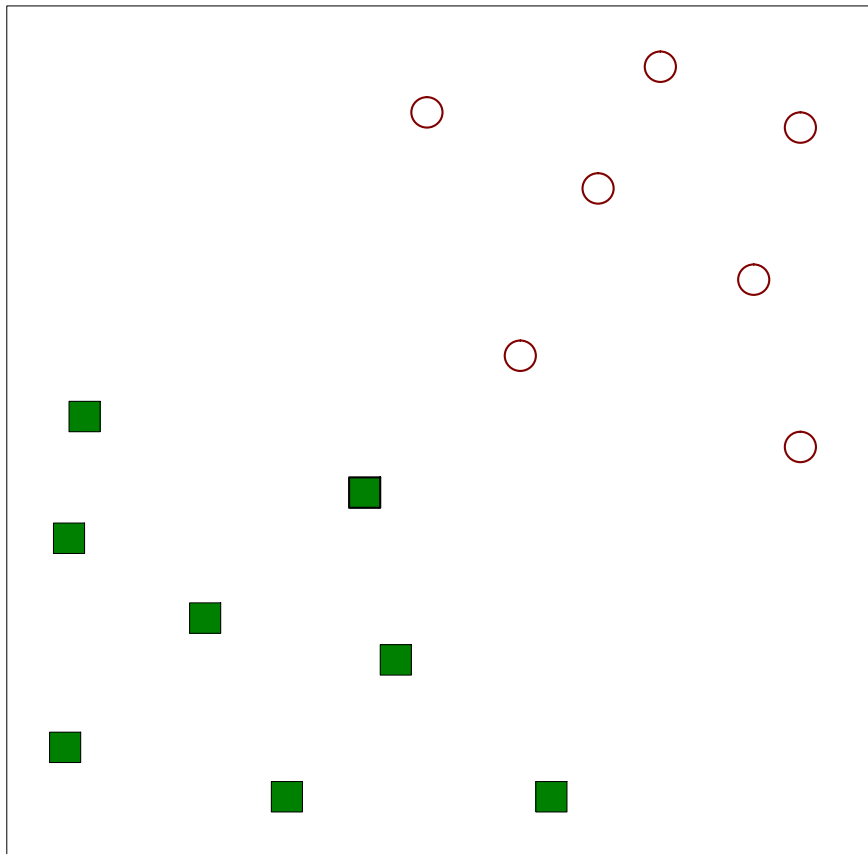
non-linearly separable classes



nonlinear decision boundaries

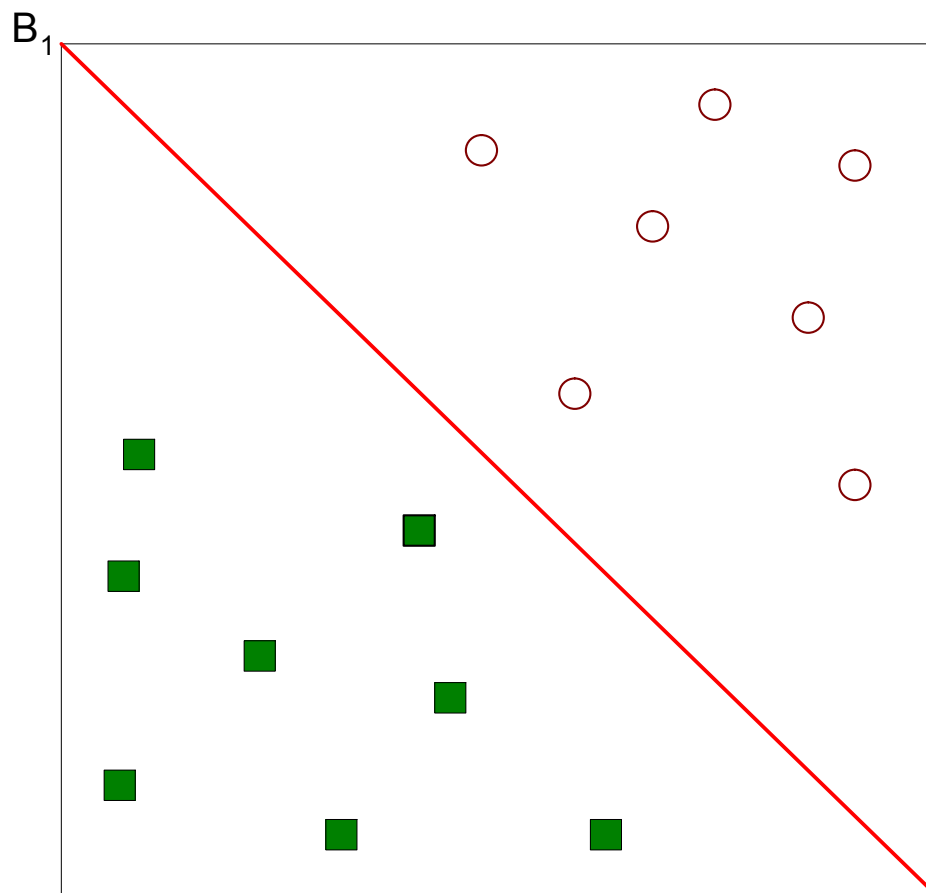
Support vector machines

Linearly
separable
classes

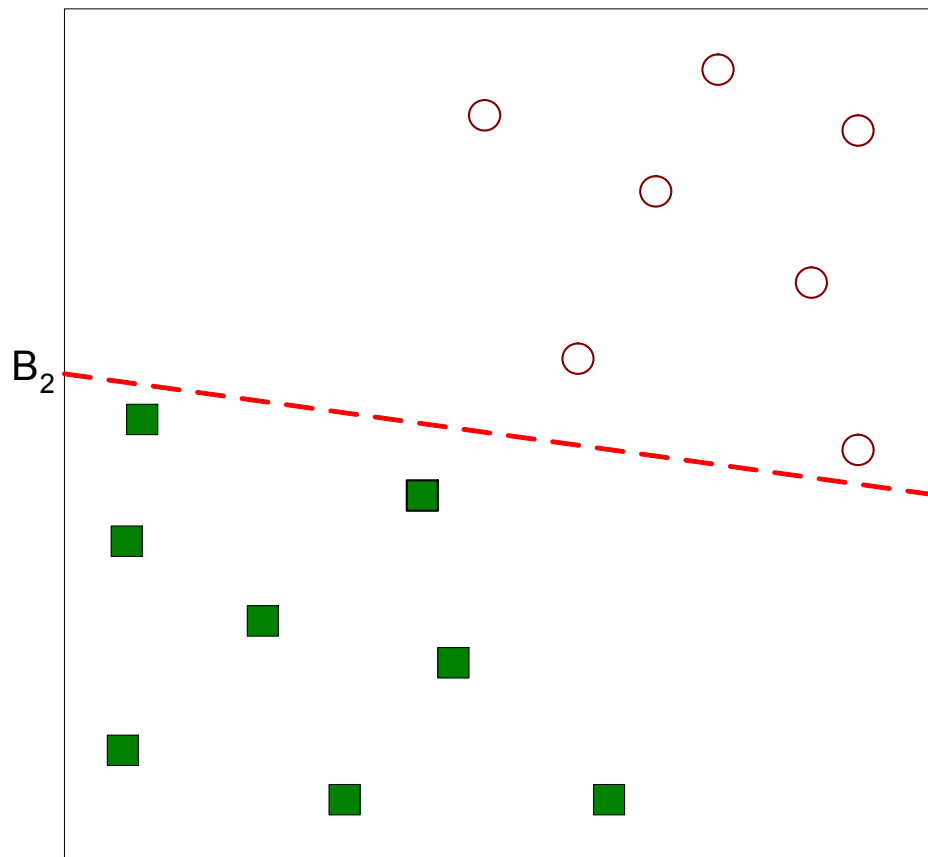


Goal: find a linear decision boundary (hyperplane)
that separates the classes

Support vector machines

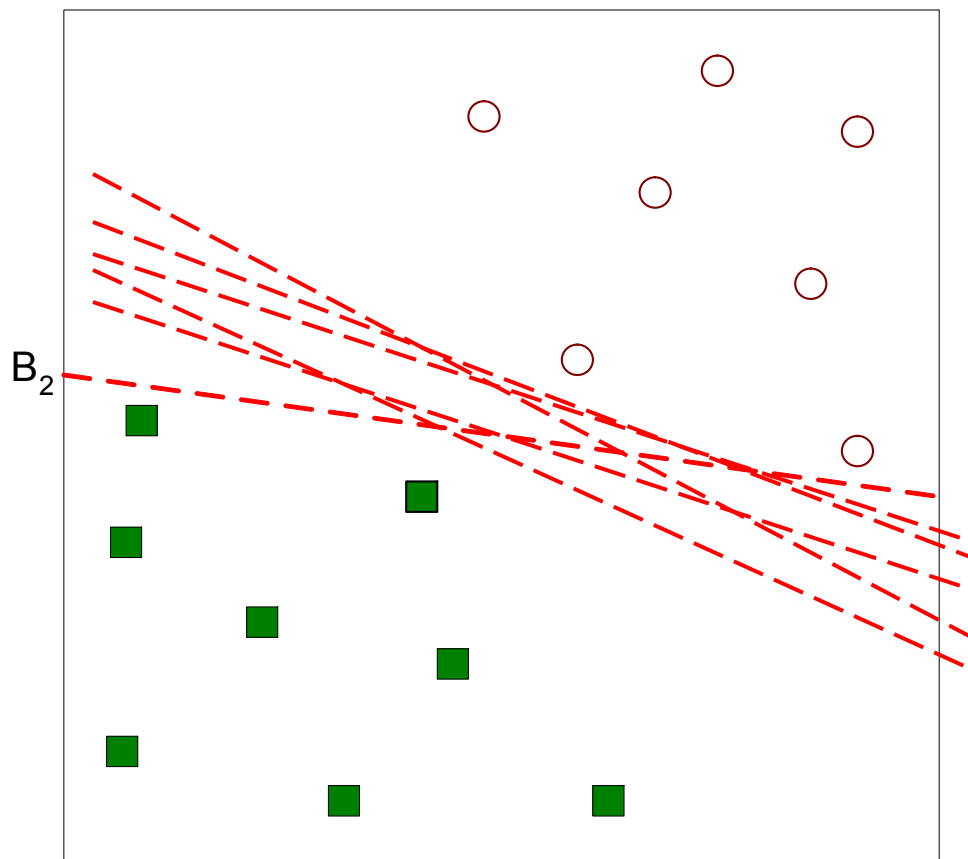


Support vector machines



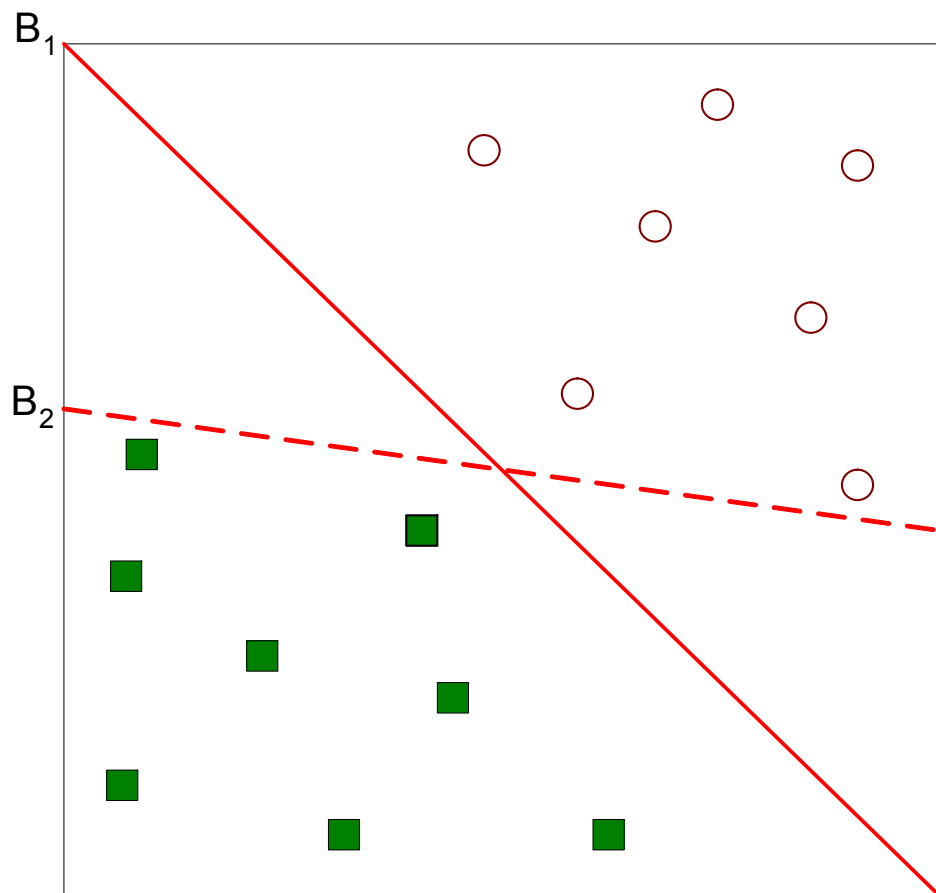
Another possible solution

Support vector machines



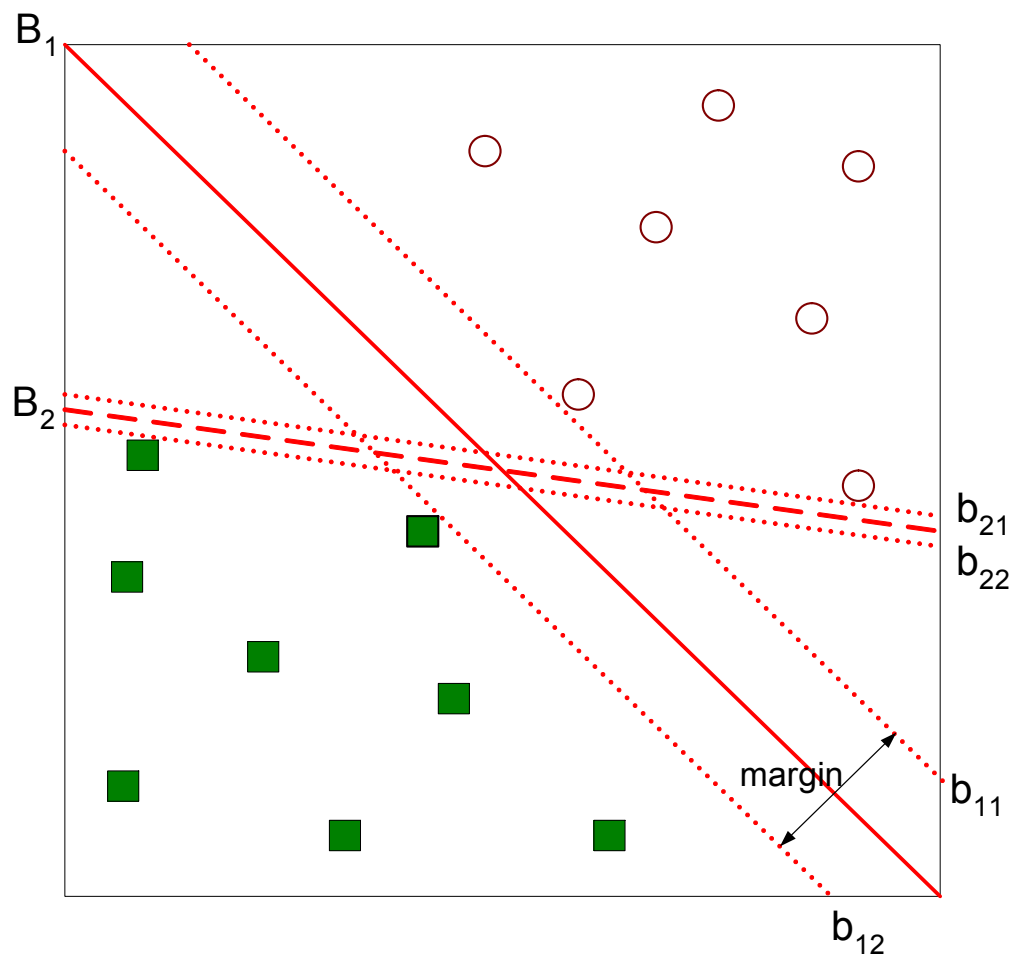
Other possible solutions

Support vector machines



Which one is better? B_1 or B_2 ? How do you define better?

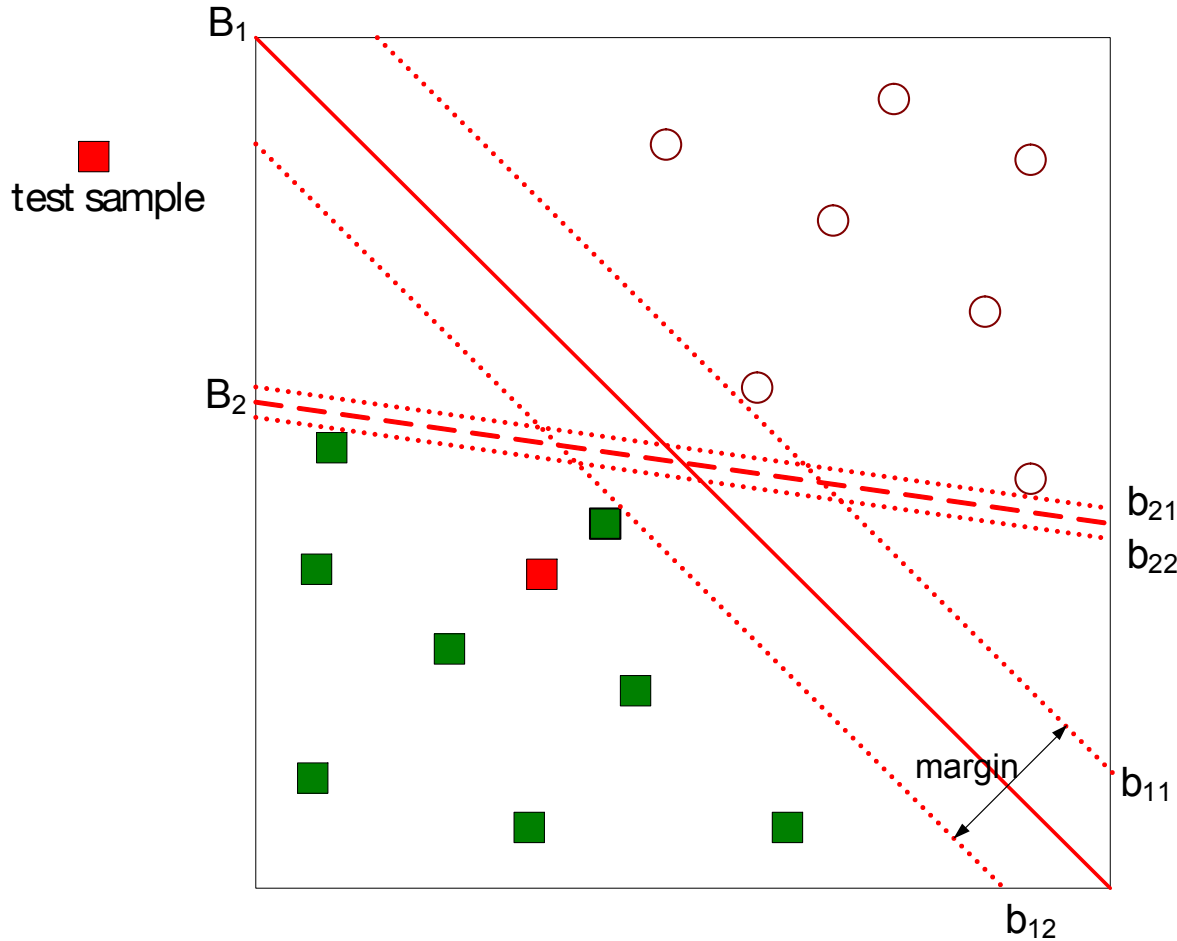
Support vector machines



Hyperplane that **maximizes** the **margin** will have better generalization

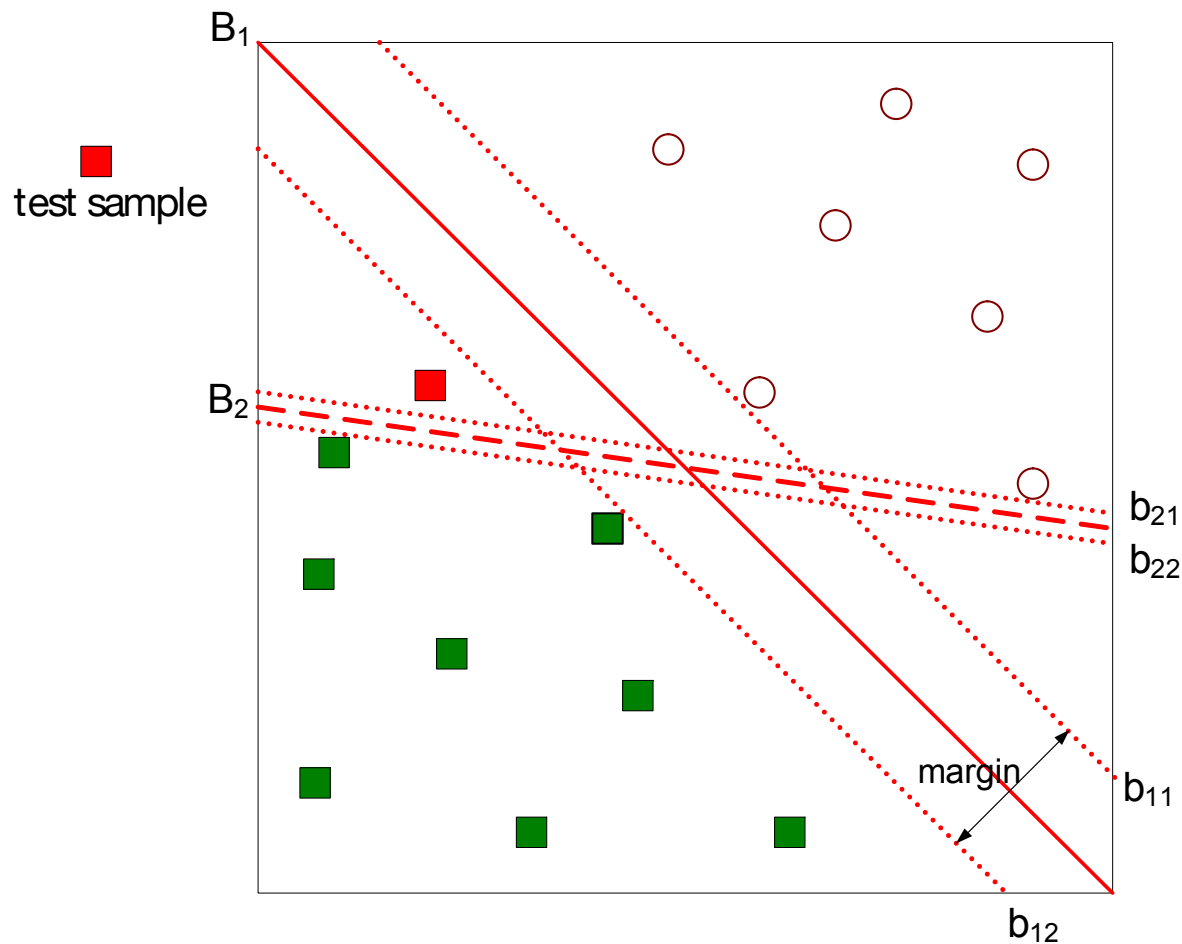
=> B1 is better than B2

Support vector machines



Hyperplane that **maximizes** the **margin** will have better generalization
=> B1 is better than B2

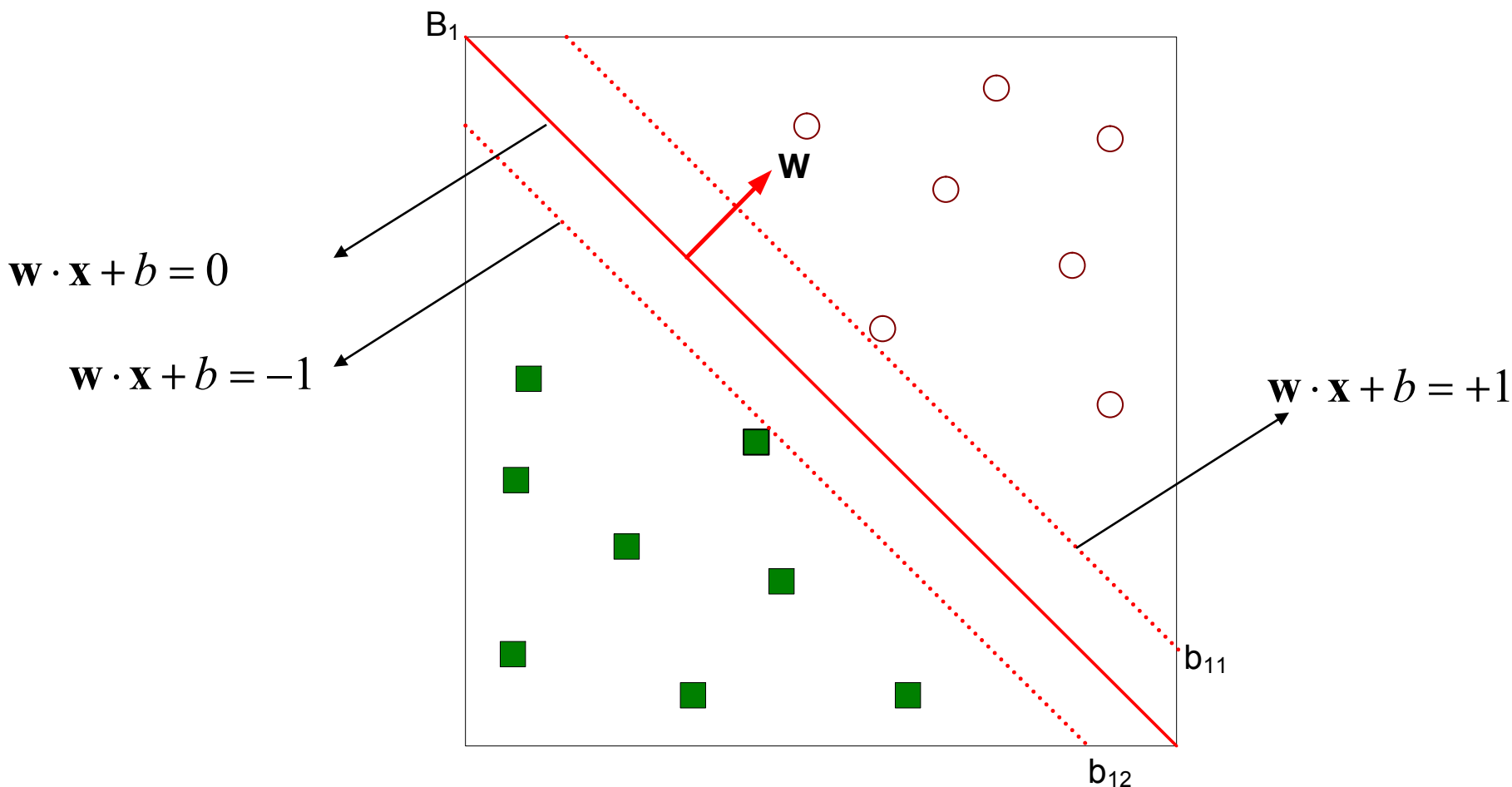
Support vector machines



Hyperplane that **maximizes** the **margin** will have better generalization

=> B_1 is better than B_2

Support vector machines



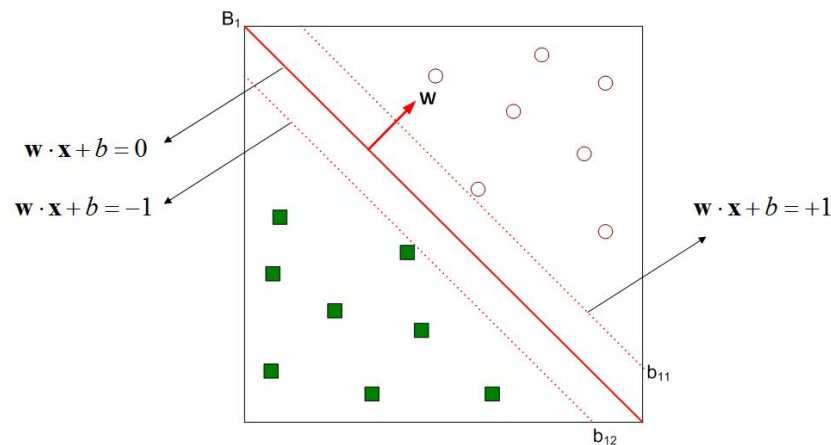
$$y_i = f(\mathbf{x}) = \begin{cases} +1 & \text{if } \mathbf{w} \cdot \mathbf{x} + b \geq 1 \\ -1 & \text{if } \mathbf{w} \cdot \mathbf{x} + b \leq -1 \end{cases}$$

Support vector machines

- Given a point x , how to calculate its distance to $w \cdot x + b = 0$?

$$\text{dis}(x_i) = \frac{|w \cdot x_i + b|}{\|w\|}$$

$$y_i = f(\mathbf{x}) = \begin{cases} +1 & \text{if } \mathbf{w} \cdot \mathbf{x} + b \geq 1 \\ -1 & \text{if } \mathbf{w} \cdot \mathbf{x} + b \leq -1 \end{cases}$$

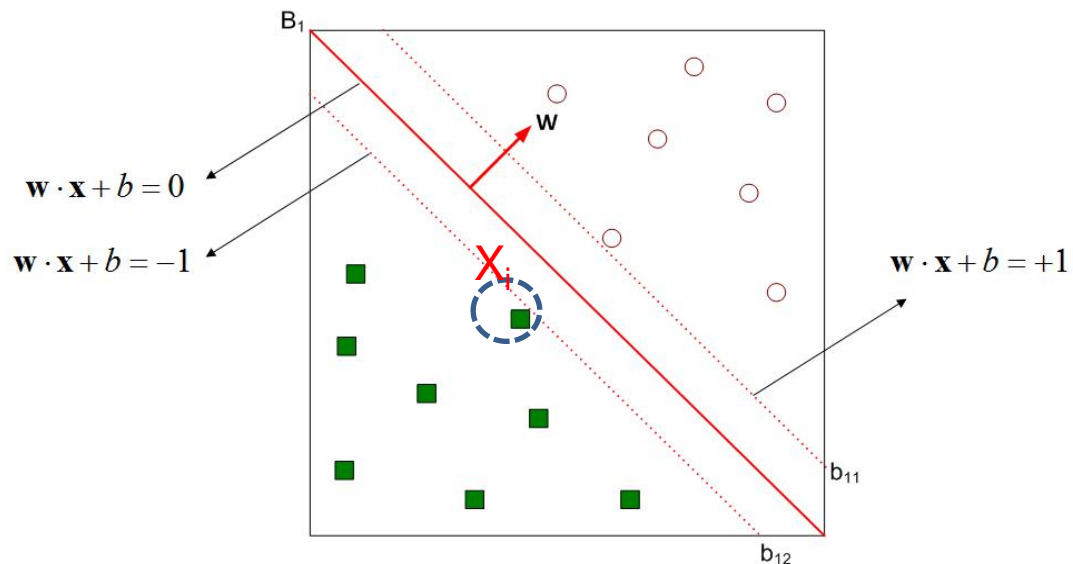


$$\text{dis}(x_i) = \frac{|w \cdot x_i + b|}{\|w\|} = \frac{y_i(w \cdot x_i + b)}{\|w\|}$$

Support vector machines

How to maximize the distance between $w \cdot x + b = 0$ and data points???

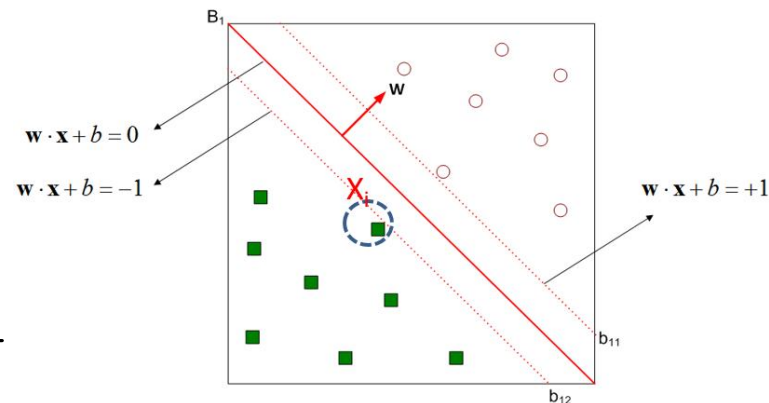
$$\text{dis}(x_i) = \frac{|w \cdot x_i + b|}{\|w\|} = \frac{1}{\|w\|}$$



What we want to do?

- (1) Maximize $\text{dis}(x_i)$;
- (2) Considering the sign of all the samples

Support vector machines



- We want to maximize: $\frac{2}{\|\mathbf{w}\|}$
- Which is equivalent to minimizing: $L(\mathbf{w}) = \frac{\|\mathbf{w}\|^2}{2}$
- But subject to the following constraints:

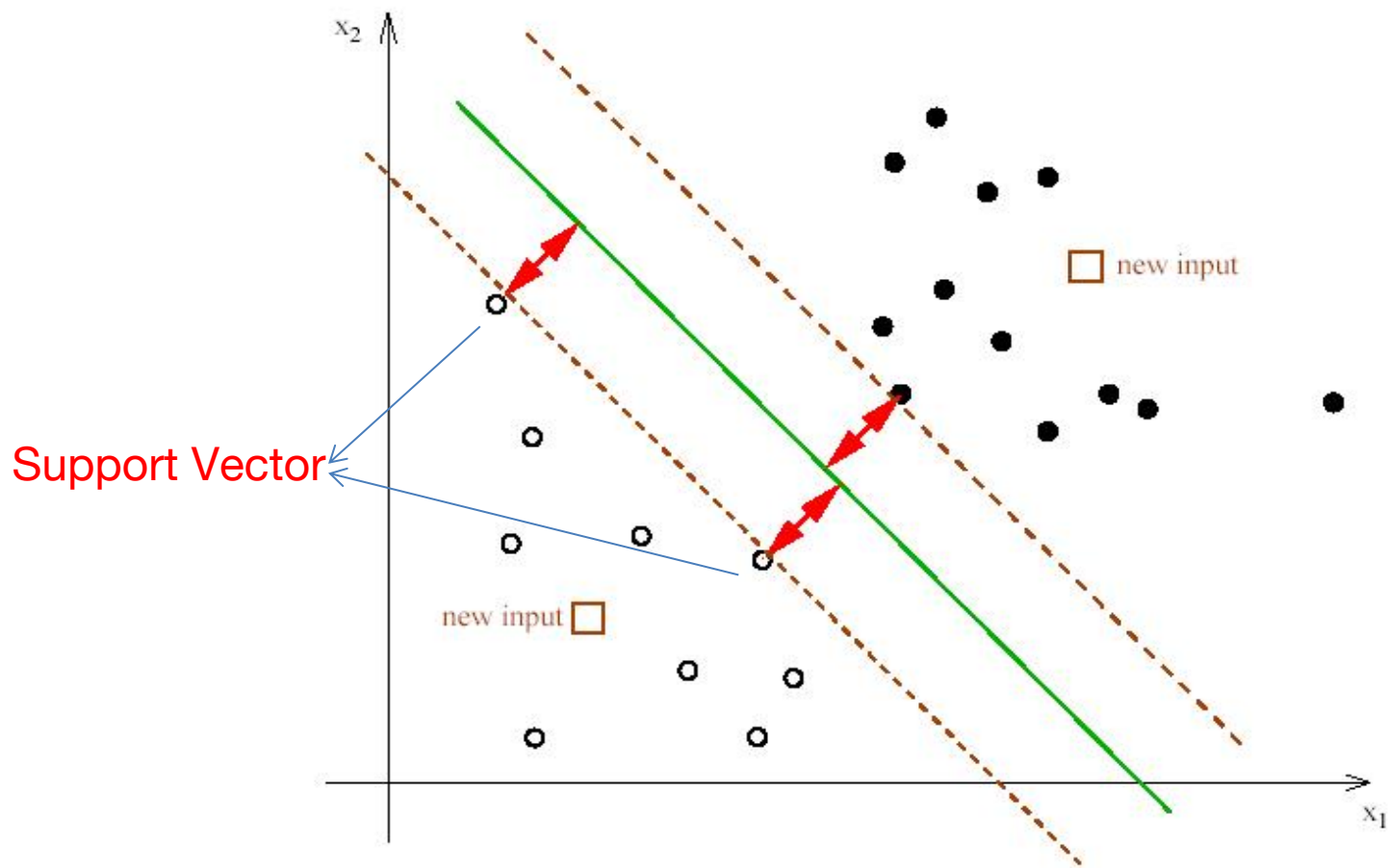
$$y_i = f(\mathbf{x}) = \begin{cases} +1 & \text{if } \mathbf{w} \cdot \mathbf{x} + b \geq 1 \\ -1 & \text{if } \mathbf{w} \cdot \mathbf{x} + b \leq -1 \end{cases}$$



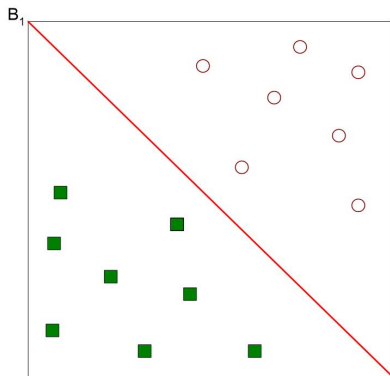
$$\text{s.t. } y_i (w x_i + b) - 1 \geq 0, i=1, 2, 3, \dots$$



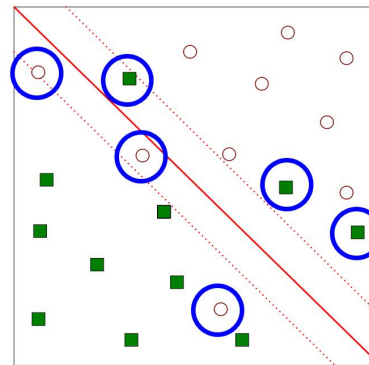
Support Vector



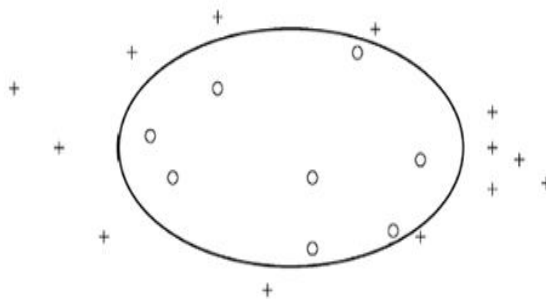
Support vector machines



linearly separable classes



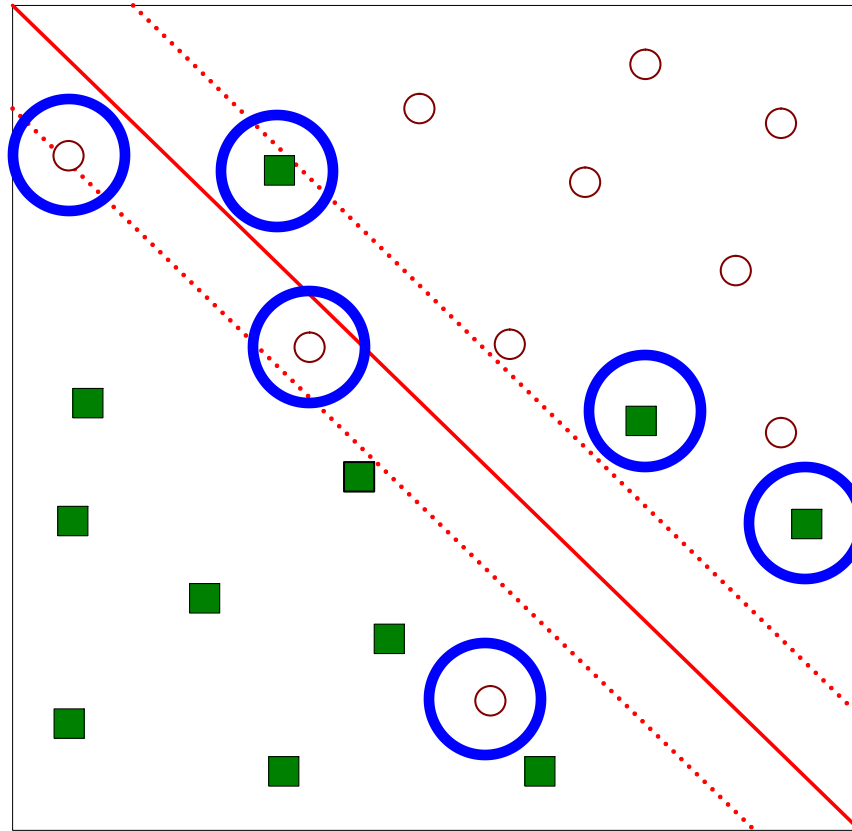
non-linearly separable classes



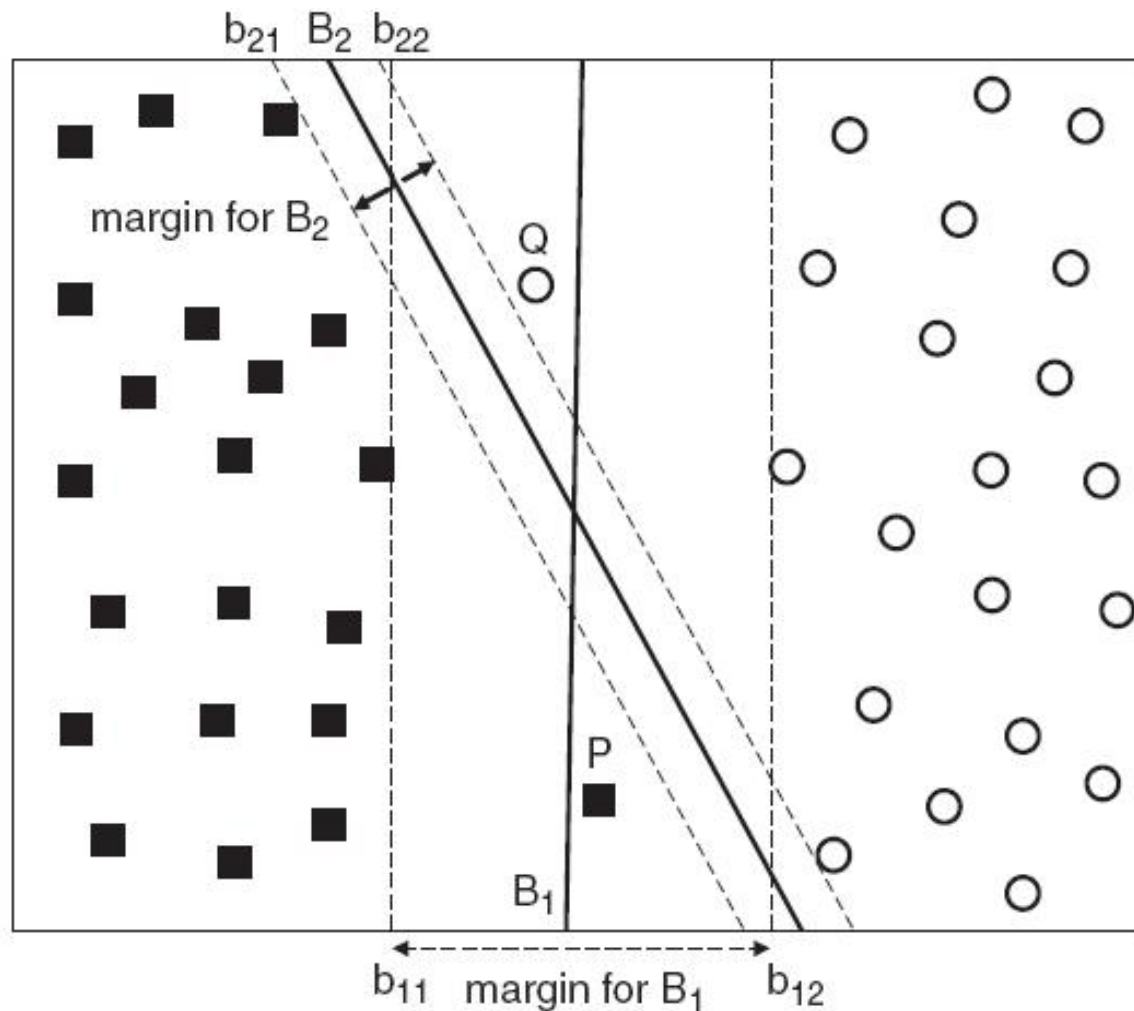
nonlinear decision boundaries

Support vector machines

What if the classes are not linearly separable?



Support vector machines



Now which one is better? B_1 or B_2 ? How do you define better?

Support vector machines

- What if the problem is not linearly separable?
- Solution: introduce slack variables

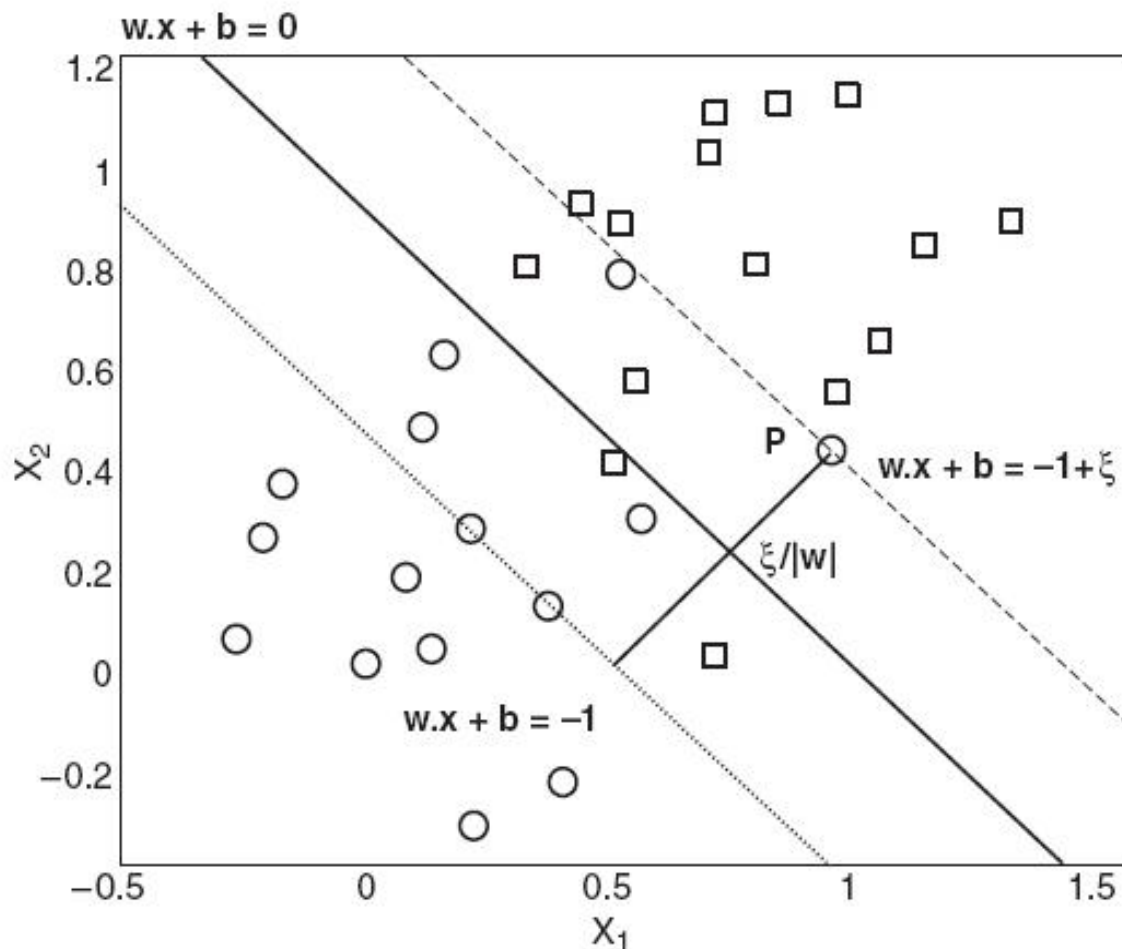
– Need to minimize:
$$L(\mathbf{w}) = \frac{\|\mathbf{w}\|^2}{2} + C \left(\sum_{i=1}^N \xi_i^k \right)$$

– Subject to:

$$y_i = f(\mathbf{x}) = \begin{cases} +1 & \text{if } \mathbf{w} \cdot \mathbf{x} + b \geq 1 + \xi_i \\ -1 & \text{if } \mathbf{w} \cdot \mathbf{x} + b \leq -1 + \xi_i \end{cases}$$

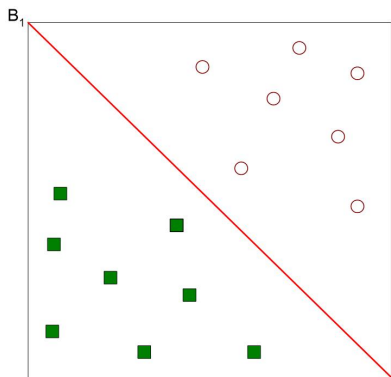
- C is an important **hyperparameter**, whose value is usually optimized by cross-validation.

Support vector machines

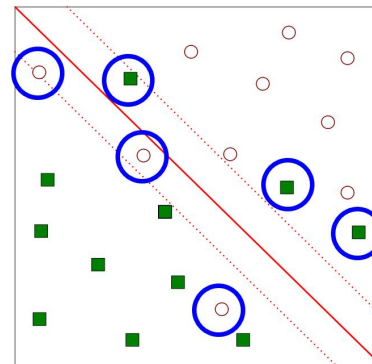


Slack variables for nonseparable data

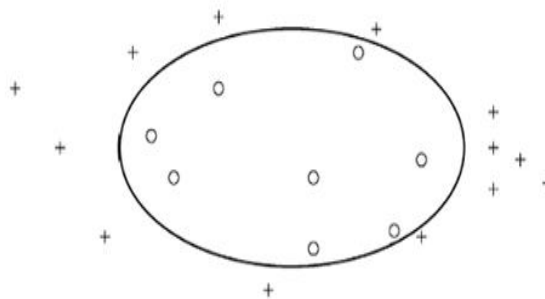
Support vector machines



linearly separable classes



non-linearly separable classes

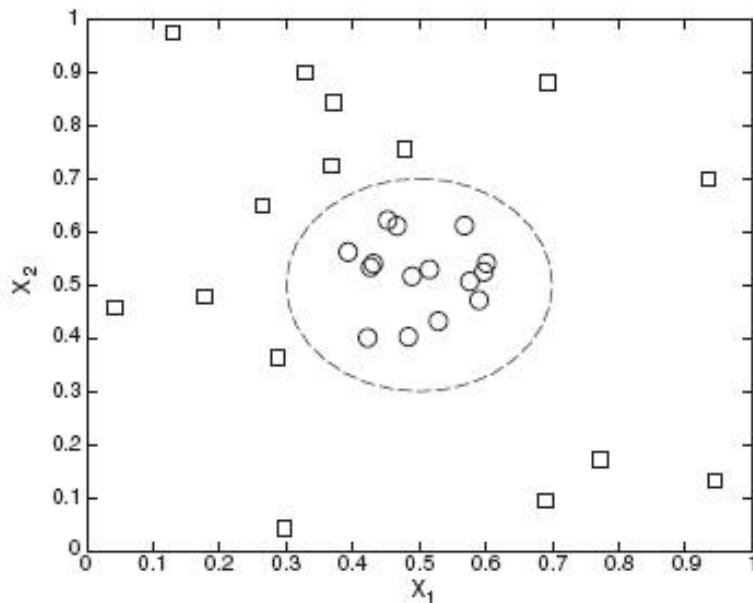


nonlinear decision boundaries

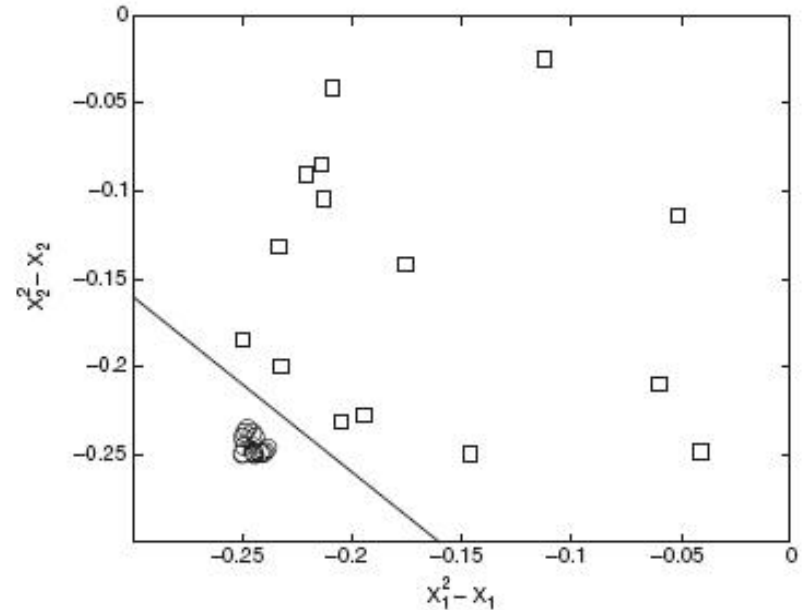
Support vector machines

Solution: nonlinear transform of attributes

$$\Phi : [x_1, x_2] \rightarrow [(x_1^2 - x_1), (x_2^2 - x_2)]$$



(a) Decision boundary in the original two-dimensional space.



(b) Decision boundary in the transformed space.

Figure 5.28. Classifying data with a nonlinear decision boundary.

Support vector machines

- Common kernel functions for SVM

- linear

$$k(\mathbf{x}_1, \mathbf{x}_2) = \mathbf{x}_1 \cdot \mathbf{x}_2$$

- polynomial

$$k(\mathbf{x}_1, \mathbf{x}_2) = (\gamma \mathbf{x}_1 \cdot \mathbf{x}_2 + c)^d$$

- Gaussian or radial basis

$$k(\mathbf{x}_1, \mathbf{x}_2) = \exp\left(-\gamma \|\mathbf{x}_1 - \mathbf{x}_2\|^2\right)$$

- sigmoid

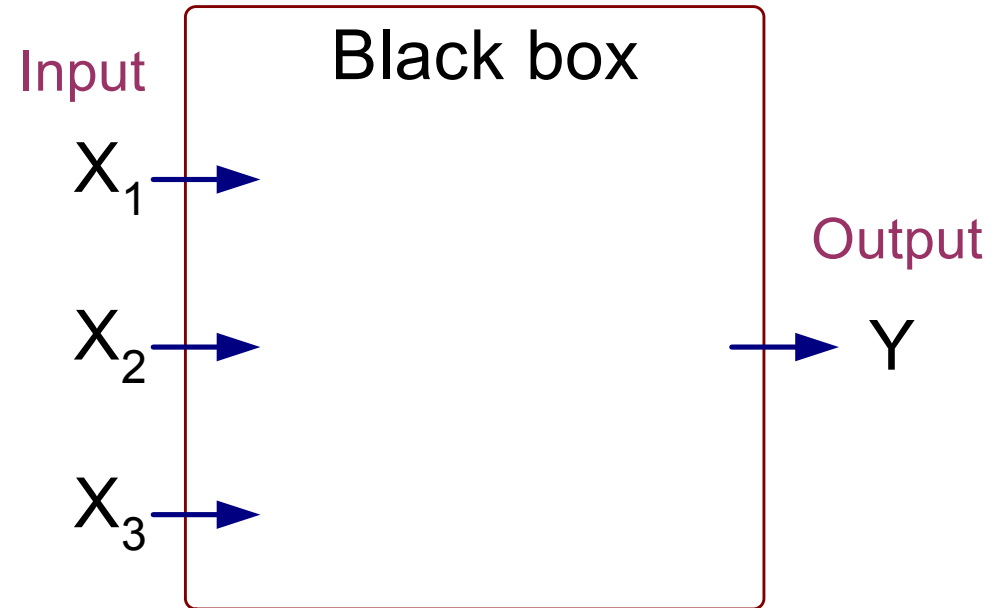
$$k(\mathbf{x}_1, \mathbf{x}_2) = \tanh(\gamma \mathbf{x}_1 \cdot \mathbf{x}_2 + c)$$

- Classification: Alternative Methods
 - SVM
 - ANN/Deep Learning
- Classification: Summarization
 - Loss Function
 - Underfitting and Overfitting
 - Model Evaluation

- **Basic Idea:** A complex non-linear function can be learned as a composition of simple processing units
- ANN is a collection of simple processing units (nodes) that are connected by directed links (edges)
 - Every node receives signals from incoming edges, performs computations, and transmits signals to outgoing edges
 - Analogous to *human brain* where nodes are neurons and signals are electrical impulses
 - Weight of an edge determines the strength of connection between the nodes
- Simplest ANN: **Perceptron** (single neuron)

Perceptron Example

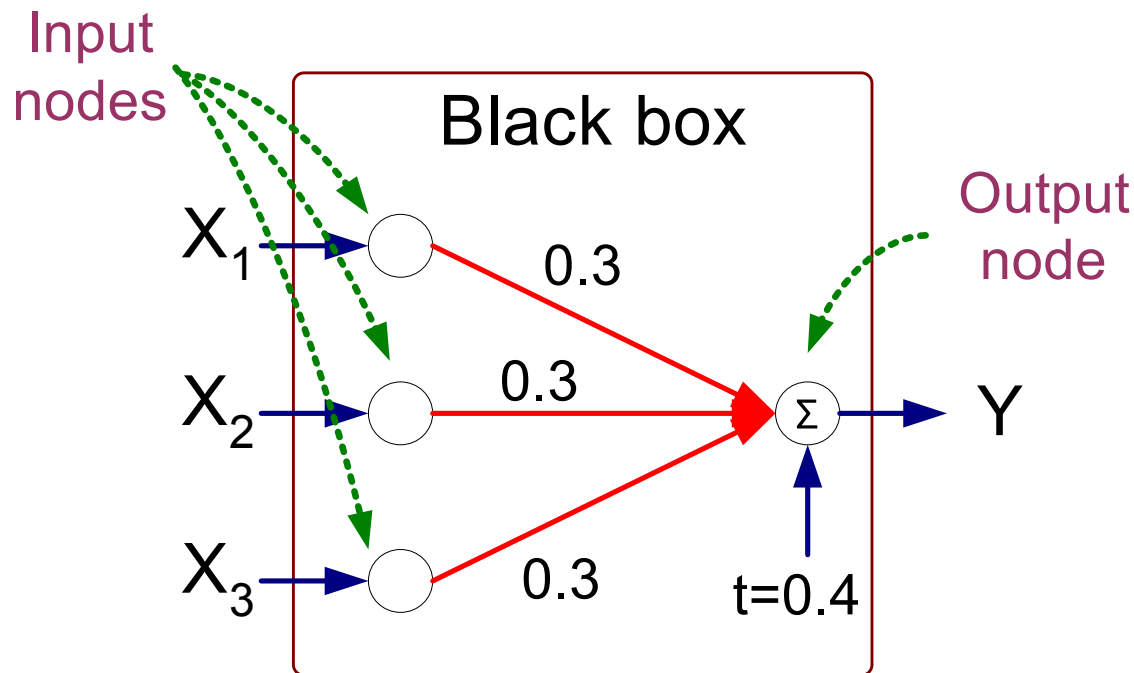
X_1	X_2	X_3	Y
1	0	0	-1
1	0	1	1
1	1	0	1
1	1	1	1
0	0	1	-1
0	1	0	-1
0	1	1	1
0	0	0	-1



Output Y is 1 if at least two of the three inputs are equal to 1.

Perceptron Example

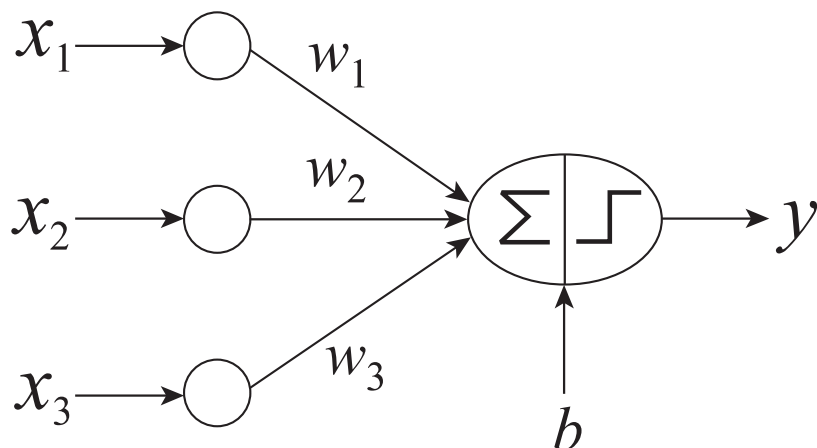
X_1	X_2	X_3	Y
1	0	0	-1
1	0	1	1
1	1	0	1
1	1	1	1
0	0	1	-1
0	1	0	-1
0	1	1	1
0	0	0	-1



$$Y = \text{sign} (0.3X_1 + 0.3X_2 + 0.3X_3 - 0.4)$$

$$\text{where } \text{sign} (x) = \begin{cases} 1 & \text{if } x \geq 0 \\ -1 & \text{if } x < 0 \end{cases}$$

Basic Architecture of Perceptron



$$y = \begin{cases} 1, & \text{if } \mathbf{w}^T \mathbf{x} + b > 0. \\ -1, & \text{otherwise.} \end{cases}$$

$$\tilde{\mathbf{w}} = (\mathbf{w}^T \ b)^T \quad \tilde{\mathbf{x}} = (\mathbf{x}^T \ 1)^T$$

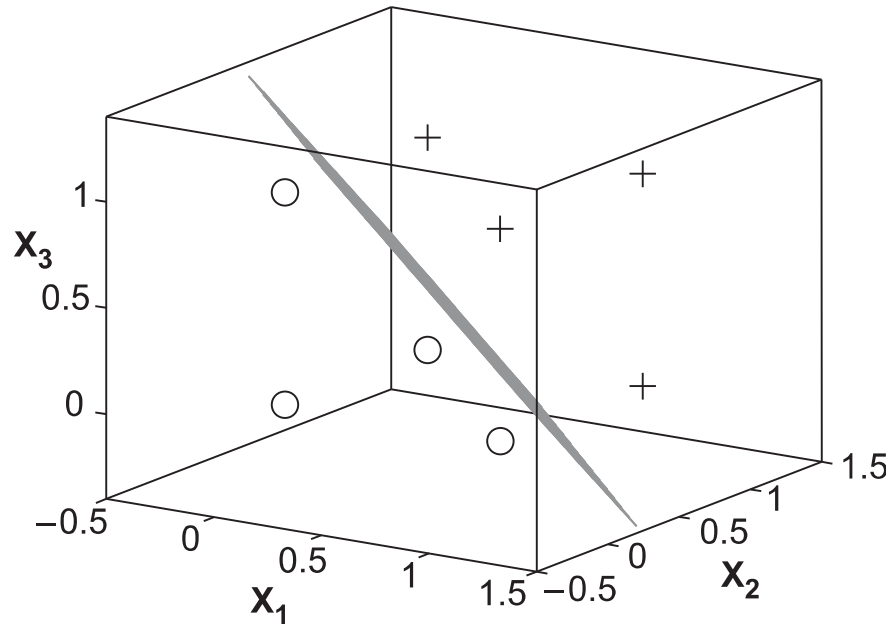
$$\hat{y} = \text{sign}(\tilde{\mathbf{w}}^T \tilde{\mathbf{x}})$$

Activation Function

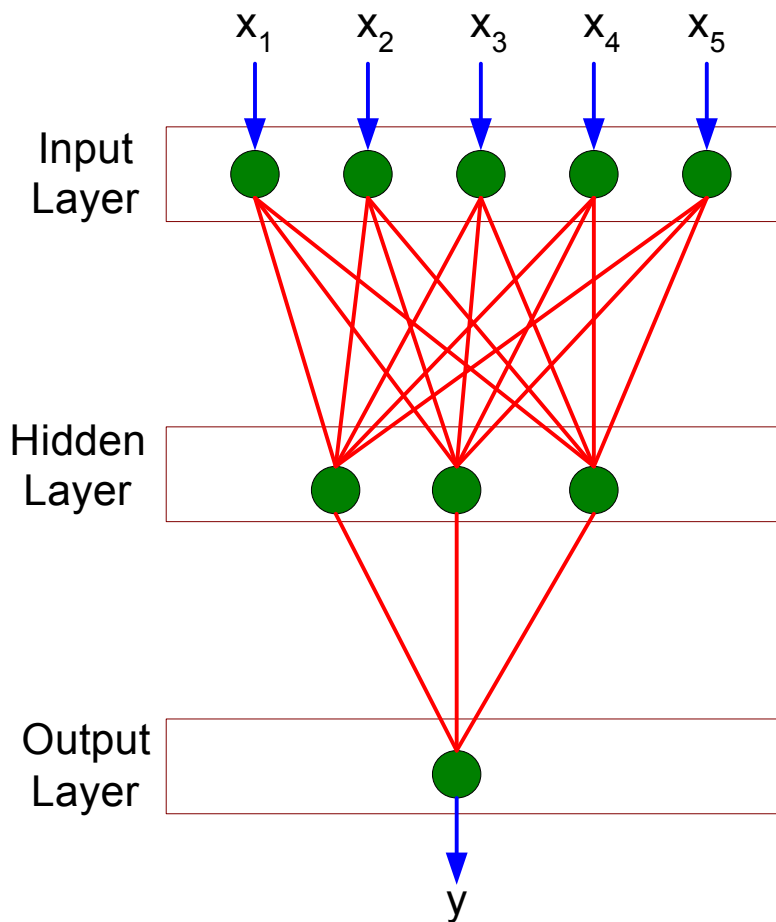
- Learns linear decision boundaries
- Related to logistic regression (activation function is sign instead of sigmoid)

Perceptron Learning

- Since y is a linear combination of input variables, decision boundary is linear



Multi-layer Neural Network



- More than one *hidden layer* of computing nodes
- Every node in a hidden layer operates on activations from preceding layer and transmits activations forward to nodes of next layer
- Also referred to as “feedforward neural networks”

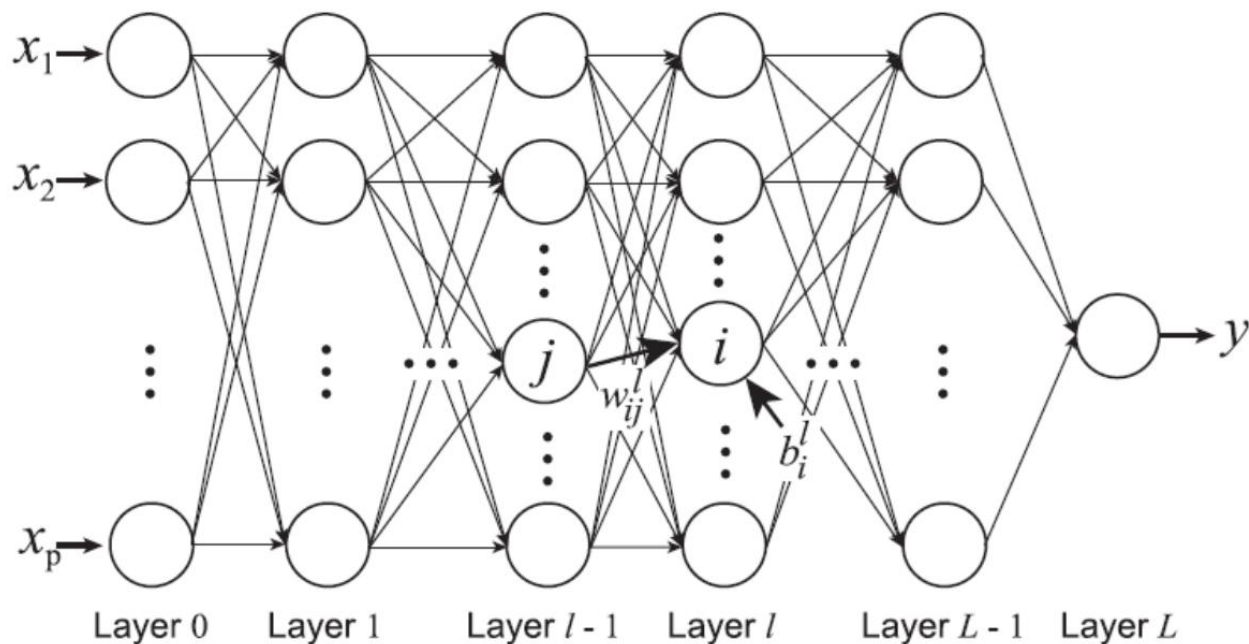
Why Multiple Hidden Layers?

- Activations at hidden layers can be viewed as features extracted as functions of inputs
- Every hidden layer represents a level of abstraction
 - *Complex features are compositions of simpler features*



- Number of layers is known as **depth** of ANN
 - *Deeper networks express complex hierarchy of features*

Multi-Layer Network Architecture



$$a_i^l = f(z_i^l) = f\left(\underbrace{\sum_j w_{ij}^l a_j^{l-1} + b_i^l}_{\text{Linear Predictor}}\right)$$

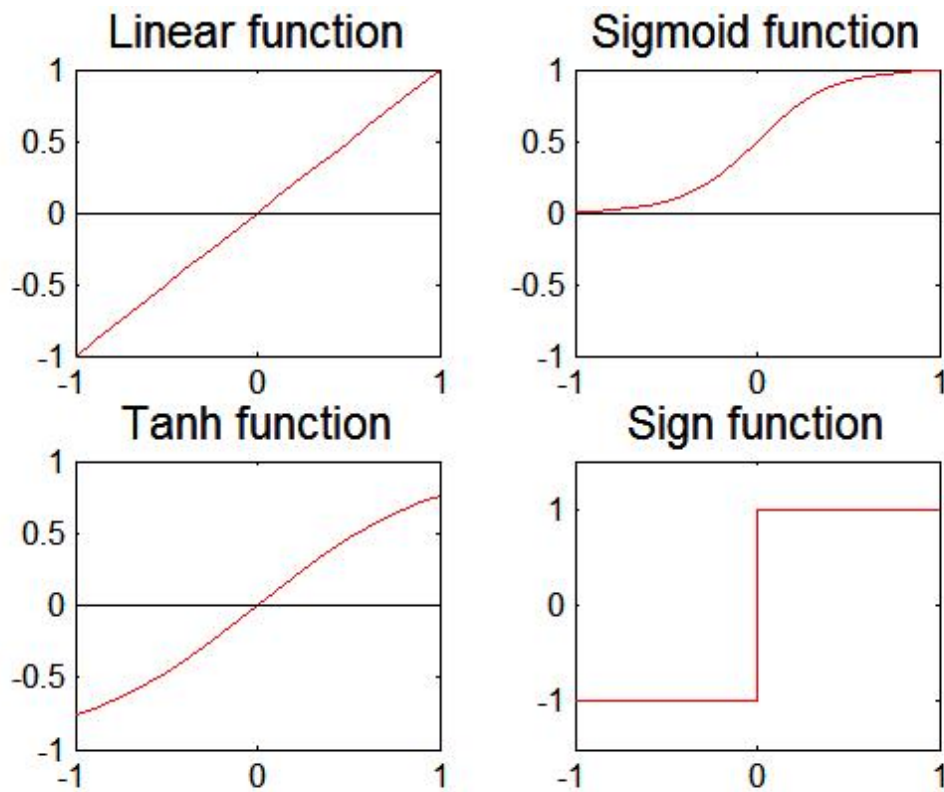
Activation value at
node i at layer l

Activation
Function

Linear Predictor

Activation Functions

$$a_i^l = f(z_i^l) = f\left(\sum_j w_{ij}^l a_j^{l-1} + b_i^l\right)$$



Design Issues in ANN

- Number of nodes in input layer
 - One input node per **binary/continuous** attribute
 - k or $\log_2 k$ nodes for each **categorical** attribute with k values
- Number of nodes in output layer
 - One output for binary class problem
 - k or $\log_2 k$ nodes for k -class problem
- Number of hidden layers and nodes per layer
- Initial weights and biases
- Learning rate, ...

- Classification: Alternative Methods
 - SVM
 - ANN/Deep Learning
- Classification: Summarization
 - Loss Function
 - Underfitting and Overfitting
 - Model Evaluation

Classification and Loss Function

Task: Given $X \in \mathcal{X}$, predict $Y \in \mathcal{Y}$.

X - test data

$\equiv$ Construct **prediction rule** $f : \mathcal{X} \rightarrow \mathcal{Y}$

Feature Space $\mathcal{X}$

Words in a document

Label Space $\mathcal{Y}$

“Sports”
“News”
“Science”
...



DJ INDU AVERAGE (DOW JONES & CO)
as of 22-Jan-2010



Share Price
“\$ 24.50”

Performance and Loss Function

Performance: $\text{loss}(Y, f(X))$

Measure of closeness between true label Y and prediction $f(X)$

$$\text{loss}(Y, f(X)) = 1_{\{f(X) \neq Y\}} \quad \text{0/1 loss}$$

$$L(Y, f(X)) = |Y - f(X)| \quad \text{Absolute Value loss}$$

$$\text{loss}(Y, f(X)) = (Y - f(X))^2 \quad \text{square loss}$$

$$L(Y, P(Y|X)) = -\log P(Y|X) \quad \text{logarithmic loss function}$$



True vs. Empirical Risk

True Risk: Target performance measure

Classification – Probability of misclassification $P(f(X) \neq Y)$

Regression – Mean Squared Error $\mathbb{E}[(f(X) - Y)^2]$

performance on a random test point (X, Y)

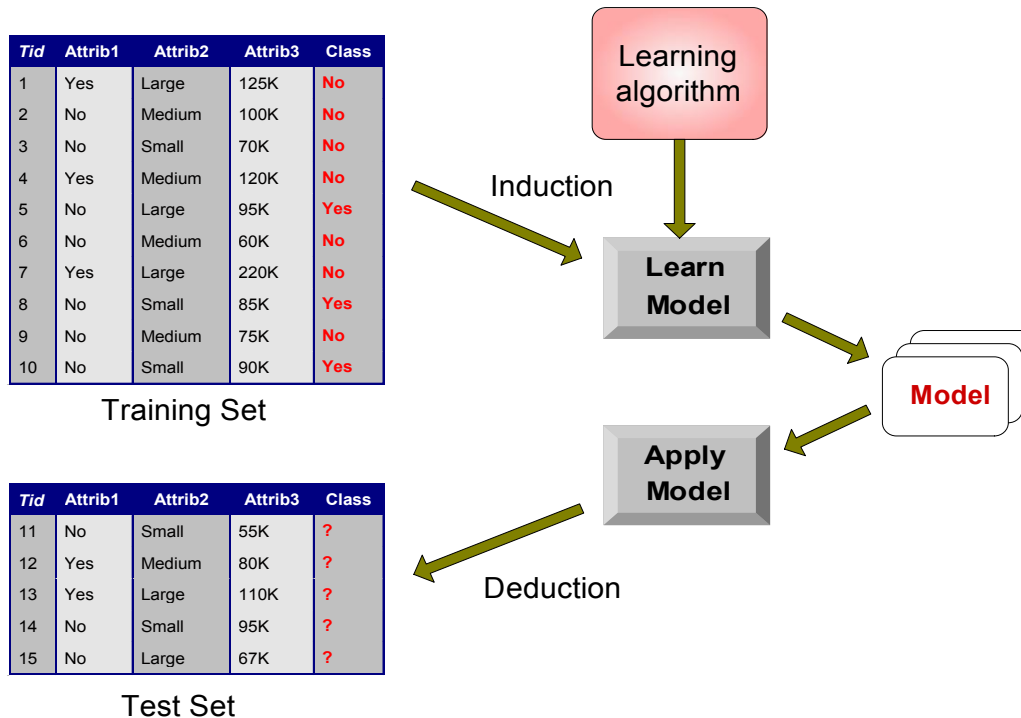
Empirical Risk: Performance on training data

Classification – Proportion of misclassified examples $\frac{1}{n} \sum_{i=1}^n 1_{f(X_i) \neq Y_i}$

Regression – Average Squared Error $\frac{1}{n} \sum_{i=1}^n (f(X_i) - Y_i)^2$

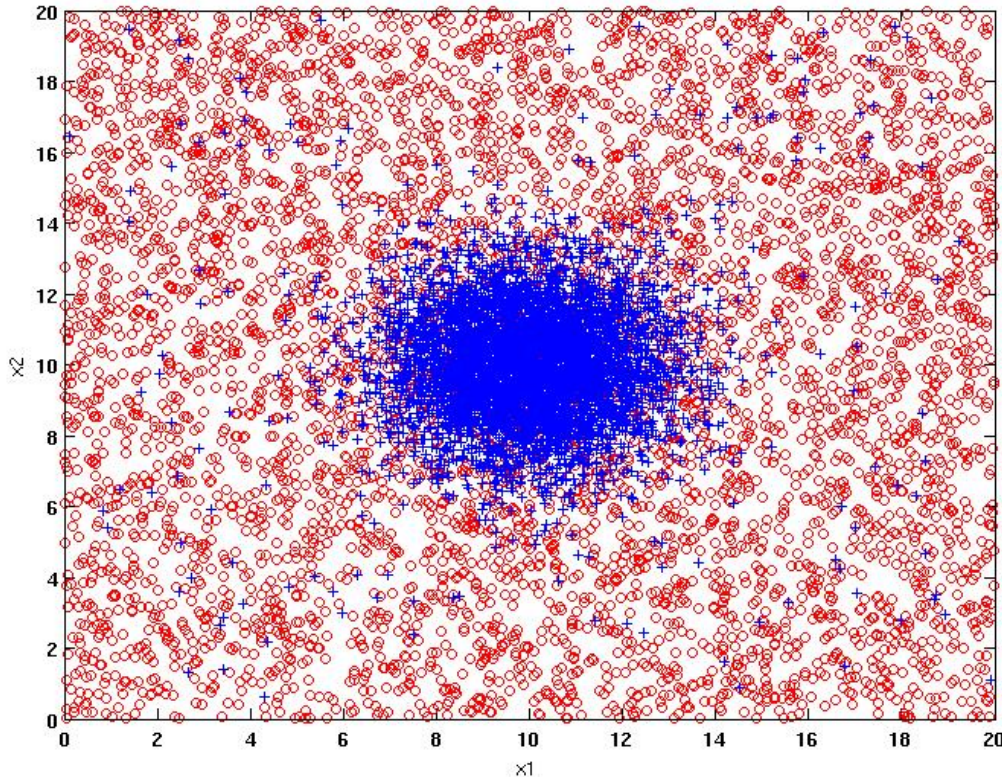
Classification Errors

- **Training errors:** Errors committed on the training set
- **Test errors:** Errors committed on the test set
- **Generalization errors:** Expected error of a model over random selection of records from same distribution



- Classification: Alternative Methods
 - KNN
 - SVM
 - ANN/Deep Learning
- Classification: Summarization
 - Loss Function
 - Underfitting and Overfitting
 - Model Evaluation

Example



Two class problem:

+ : 5400 instances

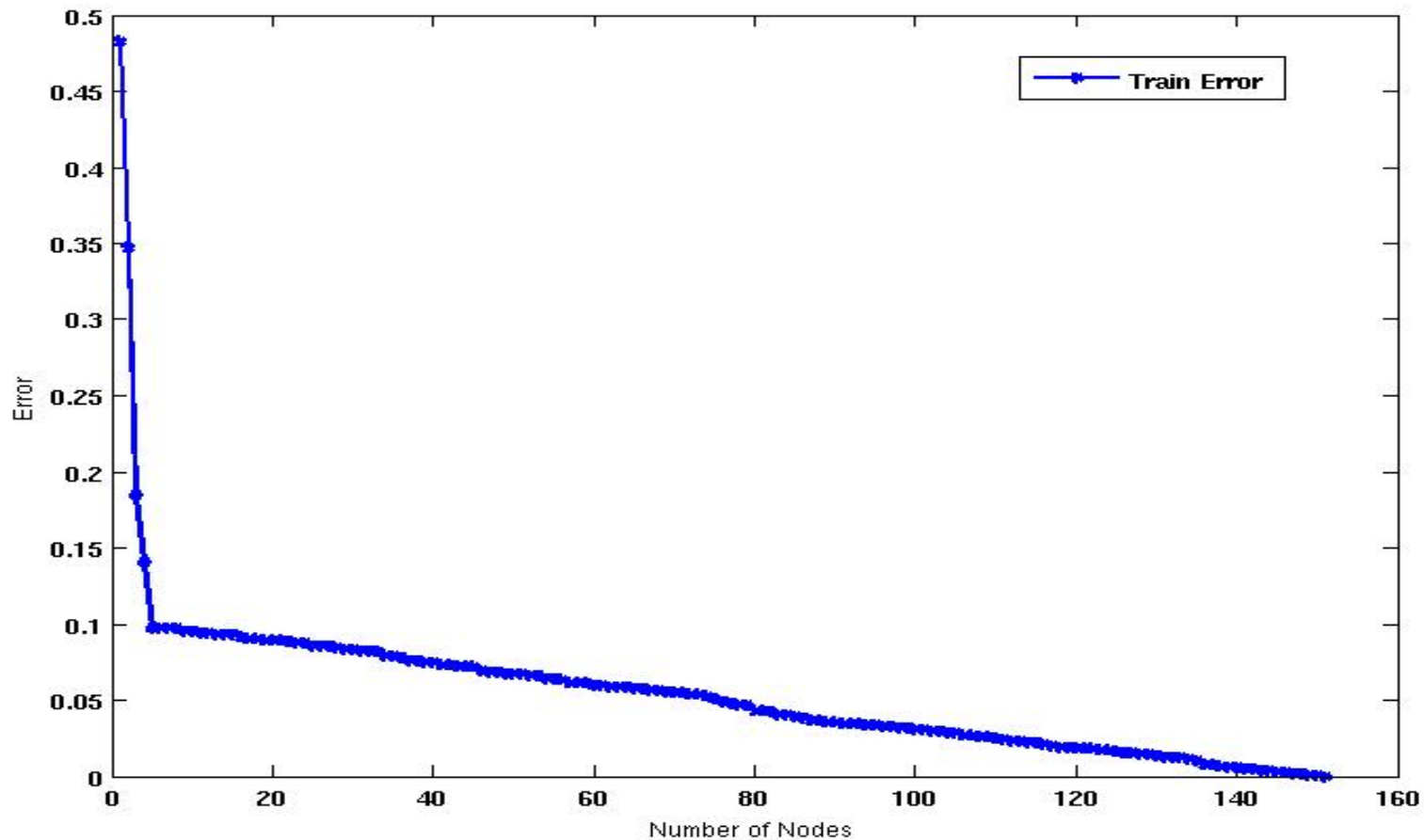
- 5000 instances generated from a Gaussian centered at (10,10)
- 400 noisy instances added

o : 5400 instances

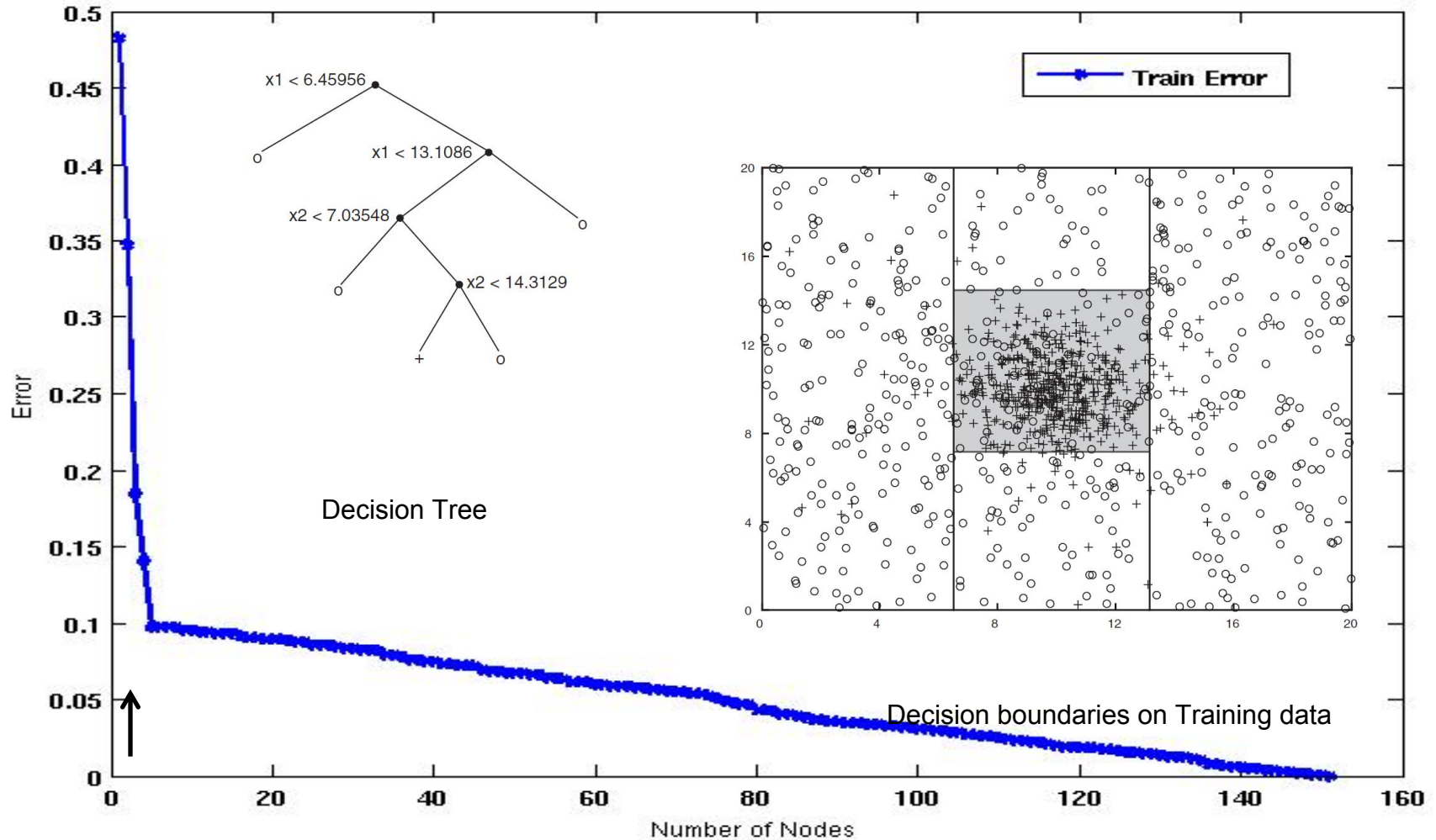
- Generated from a uniform distribution

10 % of the data used for training and 90% of the data used for testing

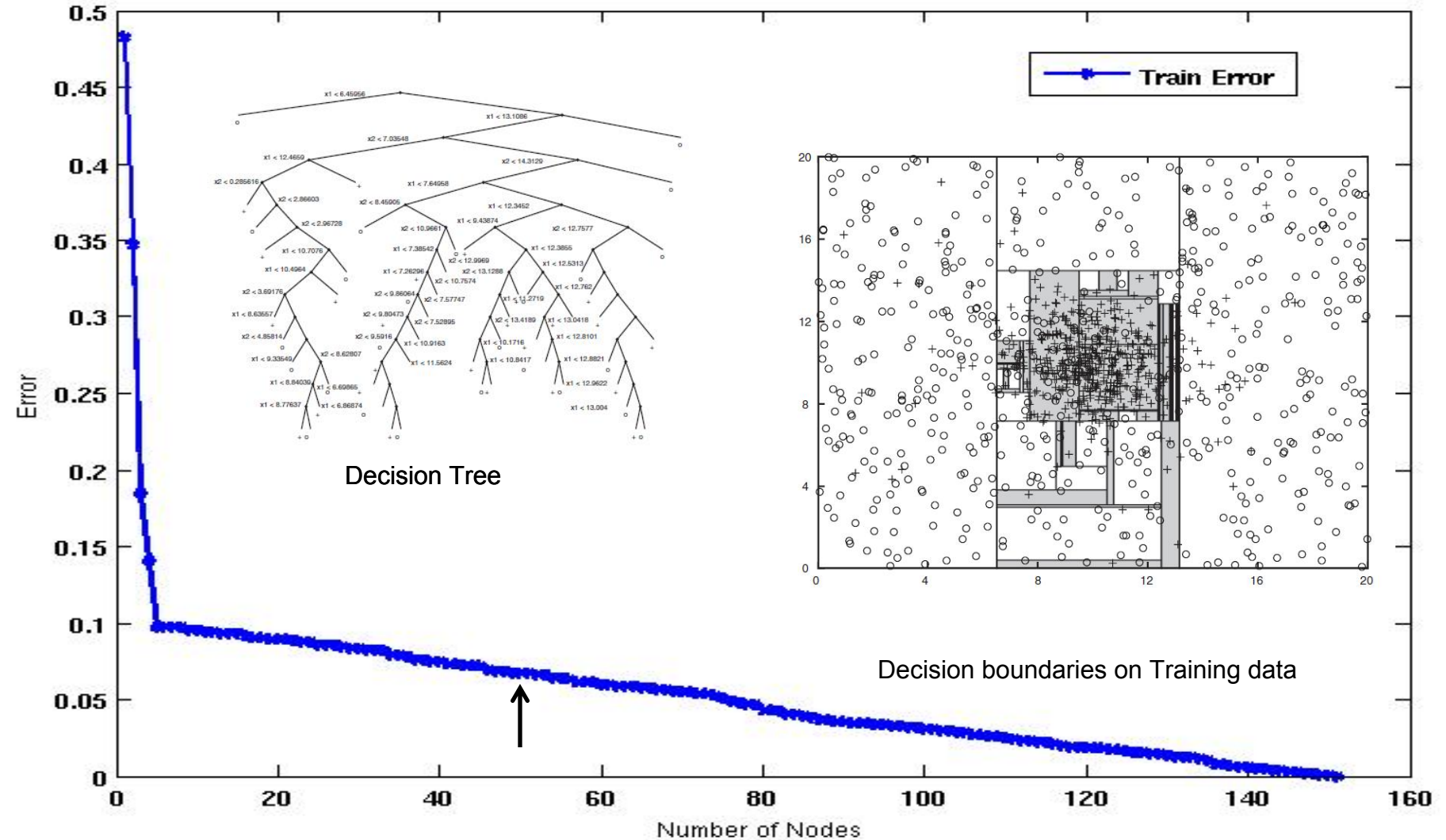
Increasing number of nodes



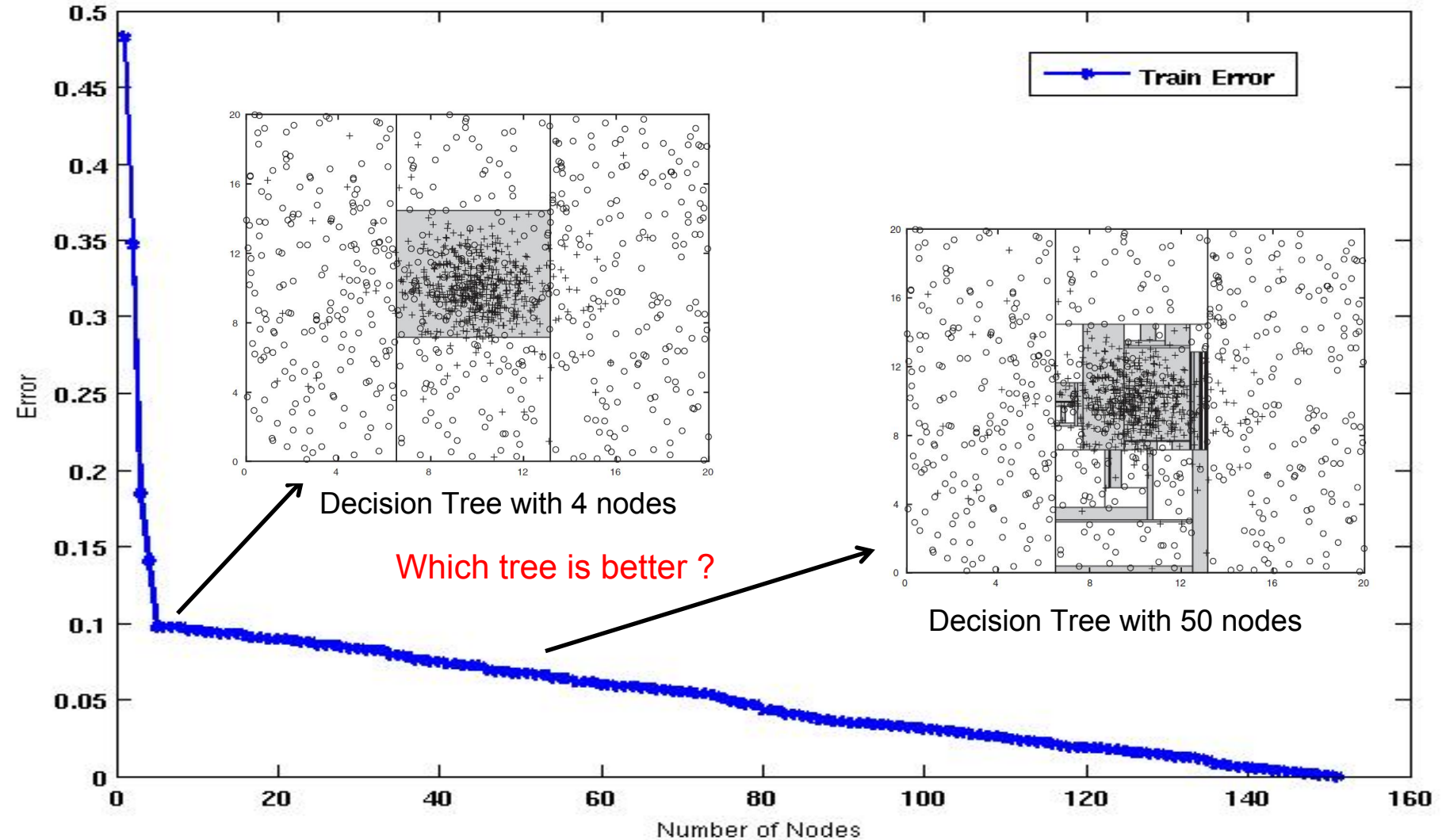
Decision Tree with 4 nodes



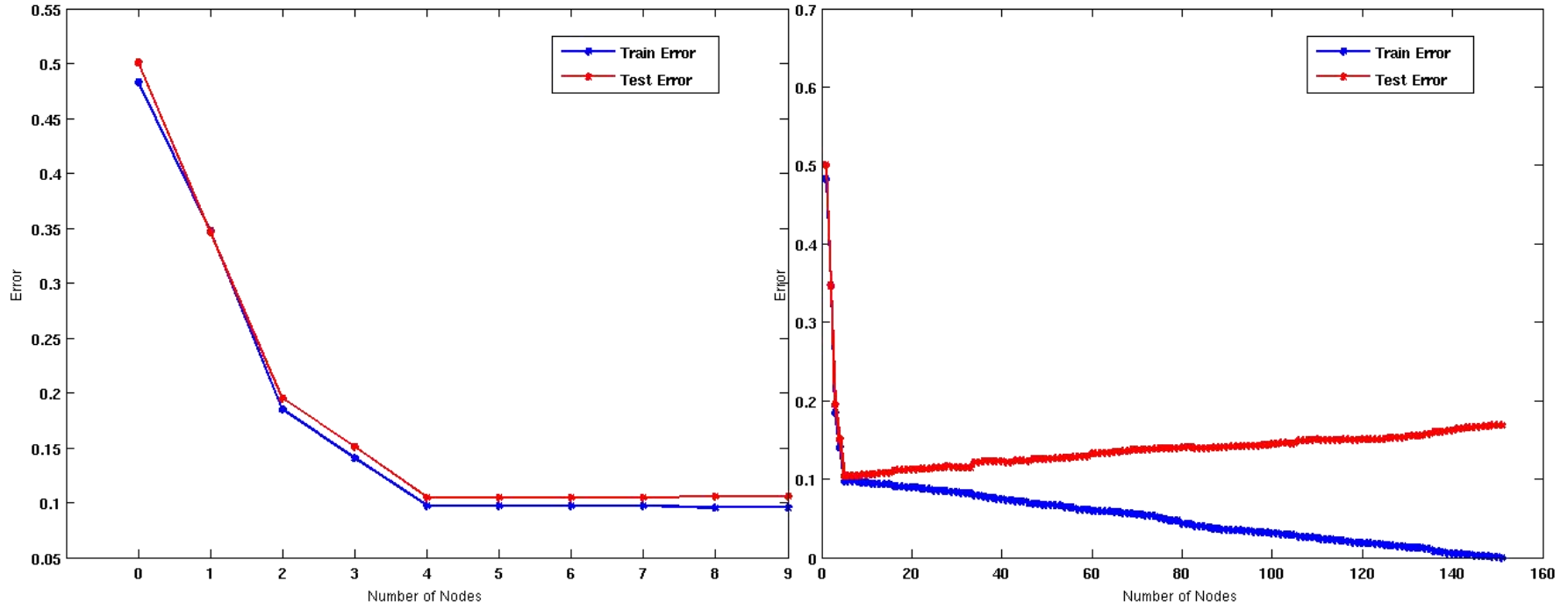
Decision Tree with 50 nodes



Which tree is better?



Model Underfitting and Overfitting



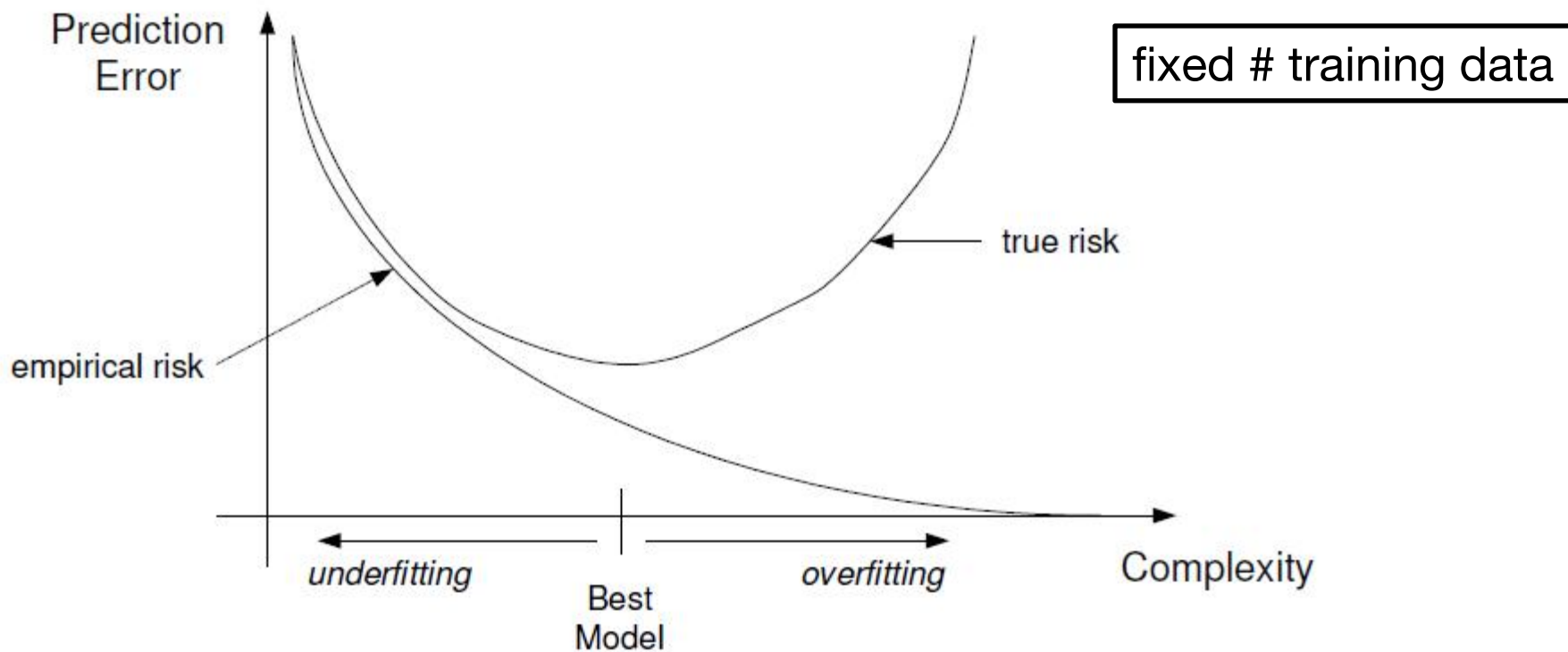
As the model becomes more and more complex, test errors can start increasing even though training error may be decreasing

Underfitting: when model is too simple, both training and test errors are large

Overfitting: when model is too complex, training error is small but test error is large

Model Complexity and Overfitting

If we allow very complicated predictors, we could overfit the training data.



Examples of Model Complexity

- **Model Spaces with increasing complexity:**

- Nearest-Neighbor classifiers with varying neighborhood sizes $k = 1, 2, 3, \dots$

Small neighborhood $\Rightarrow$ Higher complexity

- Decision Trees with depth k or with k leaves

Higher depth/ More # leaves $\Rightarrow$ Higher complexity

- Regression with polynomials of order $k = 0, 1, 2, \dots$

Higher degree $\Rightarrow$ Higher complexity

- Classification: Alternative Methods
 - SVM
 - ANN/Deep Learning
- Classification: Summarization
 - Loss Function
 - Underfitting and Overfitting
 - Evaluation

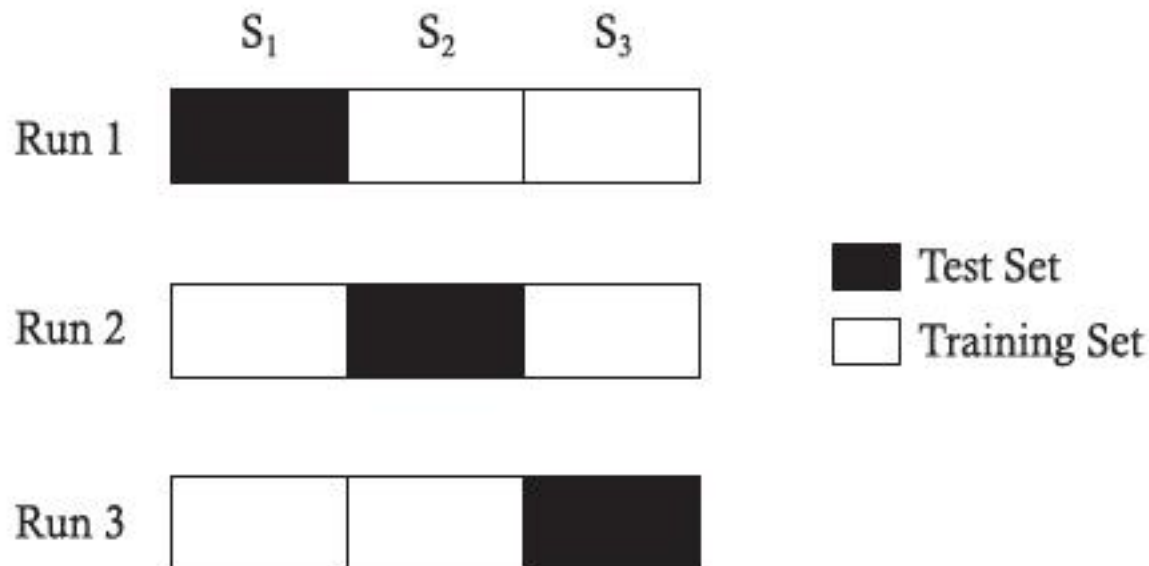


Model Evaluation

- Purpose:
 - To estimate performance of classifier on previously unseen data (test set)
- Holdout
 - Reserve $k\%$ for training and $(100-k)\%$ for testing
 - Random subsampling: repeated holdout
- Cross validation
 - Partition data into k disjoint subsets
 - k -fold: train on $k-1$ partitions, test on the remaining one
 - Leave-one-out: $k=n$

Cross-validation Example

- 3-fold cross-validation

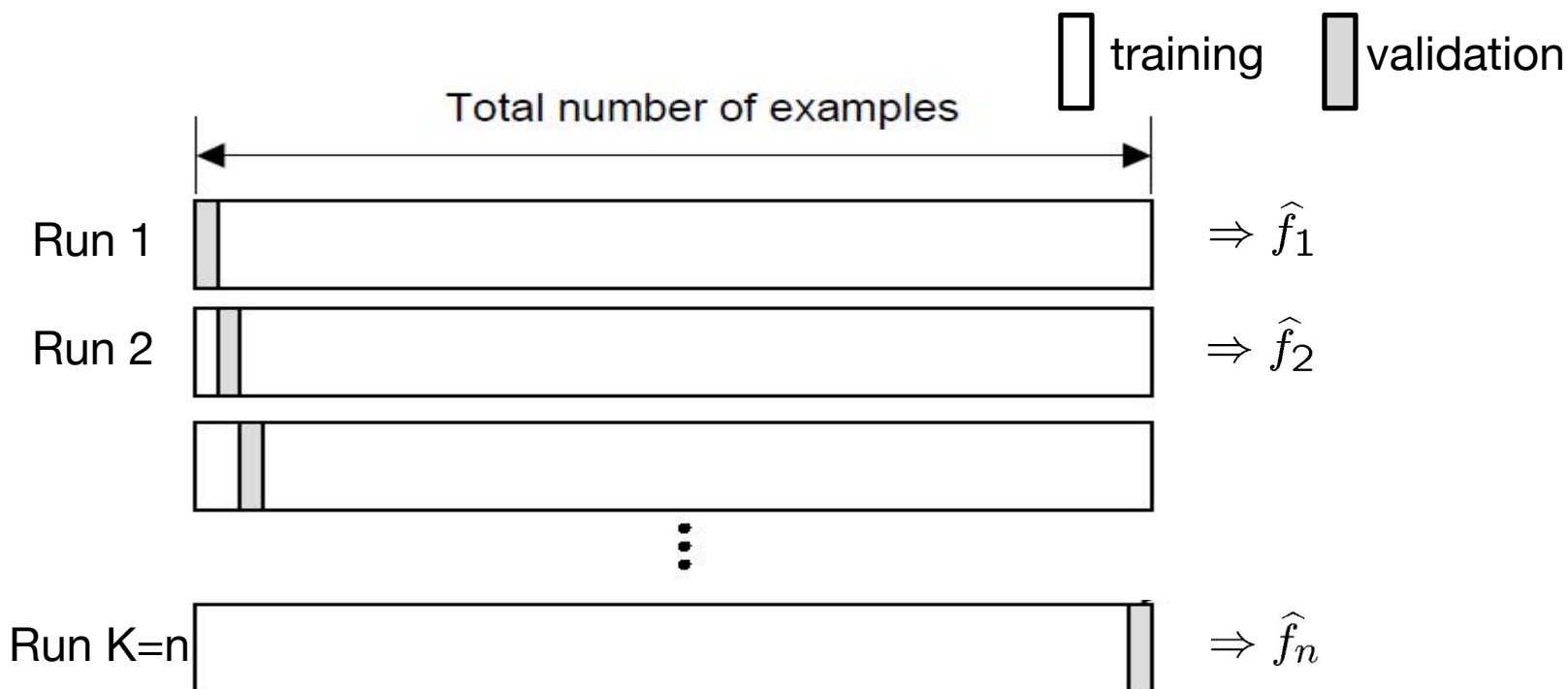


Leave-one-out

Leave-one-out (LOO) cross-validation

Special case of K-fold with $K=n$ partitions

Equivalently, train on $n-1$ samples and validate on only one sample per run for n runs



Variations on Cross-validation

- Repeated cross-validation
 - Perform cross-validation a number of times
 - Gives an estimate of the variance of the generalization error
- Stratified cross-validation
 - Guarantee the same percentage of class labels in training and test
 - Important when classes are imbalanced and the sample is small
- Use nested cross-validation approach for model selection and evaluation

How to decide the values for K ?

- Large K
 - + The bias of the error estimate will be small (many training data)
 - The variance of the error estimate will be large (few validation data)
 - The computational time will be large as well (many experiments)
- Small K
 - + The # experiments and, therefore, computation time are reduced
 - + The variance of the error estimate will be small (many validation data)
 - The bias of the error estimate will be large (few training data)
- Common choice: $K = 10$

Metrics for Model Evaluation

- Focus on the predictive capability of a model
 - Rather than how fast it takes to classify or build models, scalability, etc.
- Confusion Matrix:

ACTUAL CLASS	PREDICTED CLASS	
	Class=Yes	Class=No
	Class=Yes	Class=No
	Class=Yes	Class=No
	Class=Yes	Class=No
	Class=No	Class=No

a: TP (true positive)

b: FN (false negative)

c: FP (false positive)

d: TN (true negative)

Metrics for Model Evaluation

	PREDICTED CLASS		
		Class=Yes	Class=No
	Class=Yes	a (TP)	b (FN)
	Class=No	c (FP)	d (TN)

- Most widely-used metric:

$$\text{Accuracy} = \frac{a + d}{a + b + c + d} = \frac{TP + TN}{TP + TN + FP + FN}$$



Imbalanced Classes: Model Metrics

- Consider a 2-class problem
 - Number of Class 0 examples = 9990
 - Number of Class 1 examples = 10
- If model predicts everything to be class 0, accuracy is $9990/10000 = 99.9\%$
 - Accuracy is misleading because model does not detect any class 1 example

Cost Matrix

	PREDICTED CLASS		
	$C(i j)$	Class=Yes	Class=No
ACTUAL CLASS	Class=Yes	$C(\text{Yes} \text{Yes})$	$C(\text{No} \text{Yes})$
	Class=No	$C(\text{Yes} \text{No})$	$C(\text{No} \text{No})$

$C(i|j)$: Cost of misclassifying class j example as class i

Computing Cost of Classification

Cost Matrix	PREDICTED CLASS		
	$C(i j)$	+	-
	+	-1	100
	-	1	0

Model M_1	PREDICTED CLASS		
ACTUAL CLASS		+	-
	+	150	40
	-	60	250

Accuracy = 80%
Cost = 3910

Model M_2	PREDICTED CLASS		
ACTUAL CLASS		+	-
	+	250	45
	-	5	200

Accuracy = 90%
Cost = 4255

Cost vs Accuracy

Count	PREDICTED CLASS		
		Class=Yes	Class=No
	Class=Yes	a	b
	Class=No	c	d

Accuracy is proportional to cost if

$$1. C(\text{Yes}|\text{No})=C(\text{No}|\text{Yes}) = q$$

$$2. C(\text{Yes}|\text{Yes})=C(\text{No}|\text{No}) = p$$

$$N = a + b + c + d$$

$$\text{Accuracy} = (a + d)/N$$

Cost	PREDICTED CLASS		
		Class=Yes	Class=No
	Class=Yes	p	q
	Class=No	q	p

$$\text{Cost} = p (a + d) + q (b + c)$$

$$= p (a + d) + q (N - a - d)$$

$$= q N - (q - p)(a + d)$$

$$= N [q - (q-p) \times \text{Accuracy}]$$

Cost-Sensitive Measures

- Precision is biased towards $C(\text{Yes}|\text{Yes})$ & $C(\text{Yes}|\text{No})$
- Recall is biased towards $C(\text{Yes}|\text{Yes})$ & $C(\text{No}|\text{Yes})$
- F-measure is biased towards all except $C(\text{No}|\text{No})$

$$\text{Weighted Accuracy} = \frac{w_1 a + w_4 d}{w_1 a + w_2 b + w_3 c + w_4 d}$$

Alternative Measures

ACTUAL CLASS	PREDICTED CLASS		
		Yes	No
	Yes	TP	FN
	No	FP	TN

α is the probability that we reject the null hypothesis when it is true. This is a Type I error or a false positive (FP).

β is the probability that we accept the null hypothesis when it is false. This is a Type II error or a false negative (FN).

$$Accuracy = \frac{TP + TN}{TP + FN + FP + TN}$$

$$ErrorRate = 1 - accuracy$$

$$Precision = Positive Predictive Value = \frac{TP}{TP + FP}$$

$$Recall = Sensitivity = TP Rate = \frac{TP}{TP + FN}$$

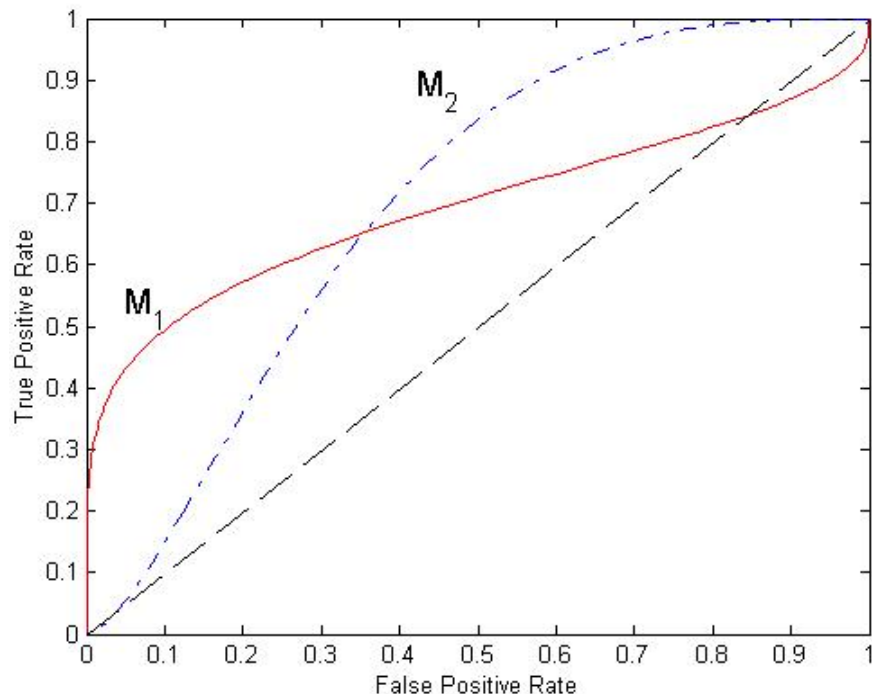
$$Specificity = TN Rate = \frac{TN}{TN + FP}$$

$$FP Rate = \alpha = \frac{FP}{TN + FP} = 1 - specificity$$

$$FN Rate = \beta = \frac{FN}{FN + TP} = 1 - sensitivity$$

$$Power = sensitivity = 1 - \beta$$

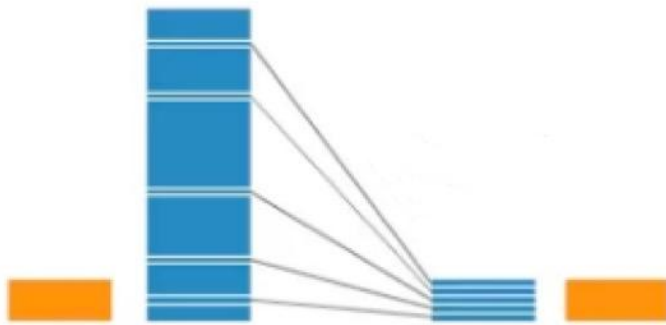
Using ROC for Model Comparison



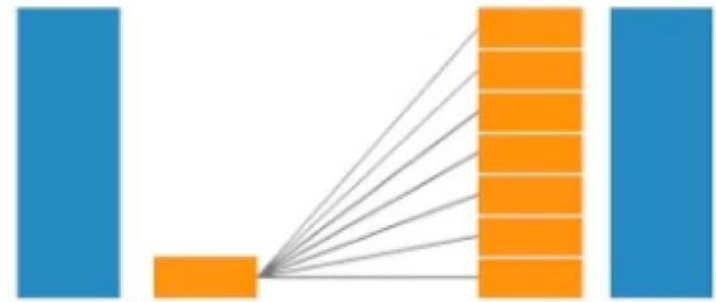
- No model consistently outperforms the other
 - M_1 is better for small FPR
 - M_2 is better for large FPR
- Area Under the ROC curve (AUC)
 - Ideal: Area = 1
 - Random guess: Area = 0.5

Imbalanced Classes

- Modify the distribution of training data so that rare class is well-represented in training set
 - Undersample the majority class
 -



Undersampling



Oversampling



Some Takeaways

- Alternative Classification Methods
- Model Selection
- Underfitting and Overfitting
- Model Evaluation

The End

■ THANK YOU

Any questions?

